# INDEX

## Android Application Development Practical

| Sr. No. | Practical / Experiment Name | Page No. | Remarks |
|---|---|---|---|
| **Module 1: Android Fundamentals and UI Design** | | | |
| 1 | Practical 1: Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation and application execution. | _____ | |
| 2 | Practical 2: Develop an Android application demonstrating programming resources (colors.xml, themes.xml, strings.xml, dimens.xml and drawable resources) and apply them in UI components. | _____ | |
| 3 | Practical 3: Develop an Android application demonstrating complete Activity Lifecycle methods and analyse execution using Logcat. | _____ | |
| 4 | Practical 4: Develop an Android application to create and integrate a Fragment and demonstrate Fragment Lifecycle methods. | _____ | |
| 5 | Practical 7: Develop an Android application using LinearLayout (horizontal and vertical) to design a structured UI. | _____ | |
| 6 | Practical 8: Develop an Android application using RelativeLayout to align UI components dynamically. | _____ | |
| 7 | Practical 9: Develop an Android application using TableLayout to design a structured login/registration form. | _____ | |
| 8 | Practical 10: Develop an Android application using ScrollView to display long scrollable content. | _____ | |
| 9 | Practical 11: Develop an Android application using AbsoluteLayout demonstrating fixed positioning. | _____ | |
| 10 | Practical 12: Develop an Android application using FrameLayout to overlay UI elements. | _____ | |
| 11 | Practical 13: Develop an Android application using ListView with ArrayAdapter and implement item click handling. | _____ | |
| 12 | Practical 14: Develop an Android application using GridView with a custom adapter and image selection handling. | _____ | |
| **Module 2: Android System Components and Data Handling** | | | |
| 13 | Practical 1: Develop an Android application implementing AppBar (Toolbar) and Options Menu. | _____ | |
| 14 | Practical 2: Develop an Android application demonstrating AlertDialog and custom DialogFragment. | _____ | |
| 15 | Practical 3: Develop an Android application implementing Contextual Menus and Popup Menus. | _____ | |
| 16 | Practical 4: Develop an Android application implementing Explicit and Implicit Intents. | _____ | |
| 17 | Practical 5: Develop an Android application demonstrating event handling using listeners (OnClick, OnLongClick, etc.). | _____ | |
| 18 | Practical 6: Develop an Android application using adapters to populate Spinner or ListView dynamically. | _____ | |
| 19 | Practical 7: Develop an Android application implementing started and bound services. | _____ | |
| 20 | Practical 8: Develop an Android application implementing notifications using Notification Manager. | _____ | |
| 21 | Practical 9: Develop an Android application implementing static and dynamic Broadcast Receivers. | _____ | |
| 22 | Practical 10: Develop an Android application implementing SQLite database CRUD operations. | _____ | |
| 23 | Practical 11: Develop an Android application integrating SQLite with ListView or RecyclerView. | _____ | |
| 24 | Practical 13: Develop an Android application demonstrating Threads and Handlers for background processing. | _____ | |
| 25 | Practical 14: Develop an Android application implementing Media APIs and Telephone APIs. | _____ | |
| 26 | Practical 15: Develop an Android application implementing runtime permissions and security handling in AndroidManifest.xml. | _____ | |

## MODULE 1: ANDROID FUNDAMENTALS AND UI DESIGN

# PRACTICAL 1

**DEVELOP A SIMPLE "HELLO WORLD" ANDROID APPLICATION DEMONSTRATING PROJECT CREATION, ANDROID STUDIO INTERFACE, ANDROID COMPONENTS, AVD CREATION AND APPLICATION EXECUTION**.

### ANDROID BASICS AND RESOURCES

### I. INSTALL ANDROID STUDIO:

**Step 1: Download Android Studio.**

Visit: https://developer.android.com/studio

**Note:**

Download the latest version to get all the help related to application development.

**Step 2: Install Android Studio**.

1. Run the installer. Click Next.

2. Select:

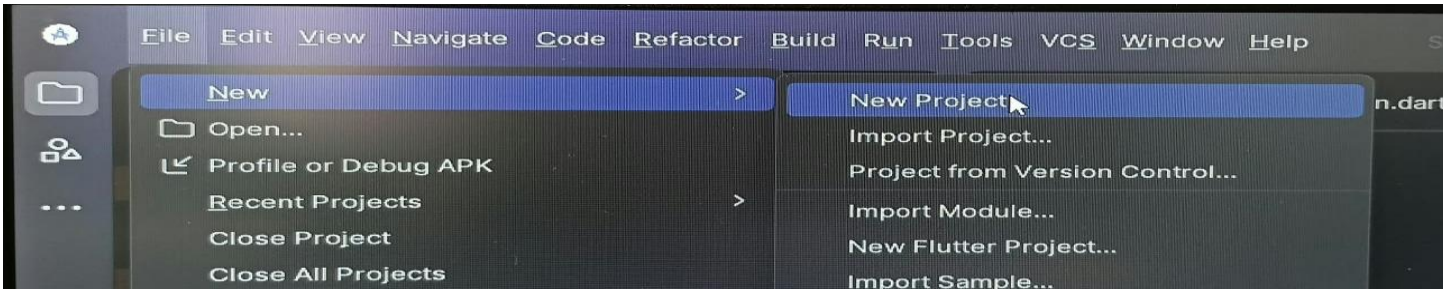
1) Android Studio.
11) Android Virtual Device.
3. Click Next. Choose installation location.
4. Click Install.
5. Wait for installation to complete.
6. Click Finish.
V. FIRST LAUNCH SETUP:
When Android Studio starts for the first time:
1. Select Standard Installation.
2. Choose Theme:
i) Light.
Car
ii) Dark.
XR

3. Accept SDK installation.

4. Wait for Gradle and SDK downloads.

**II. CREATE NEW ANDROID PROJECT**:

2. Step-by-Step Procedure:
3. Step 1: Open Android Studio.
4. Launch Android Studio.
5. Step 2: Create New Project.
6. Click: New Project.



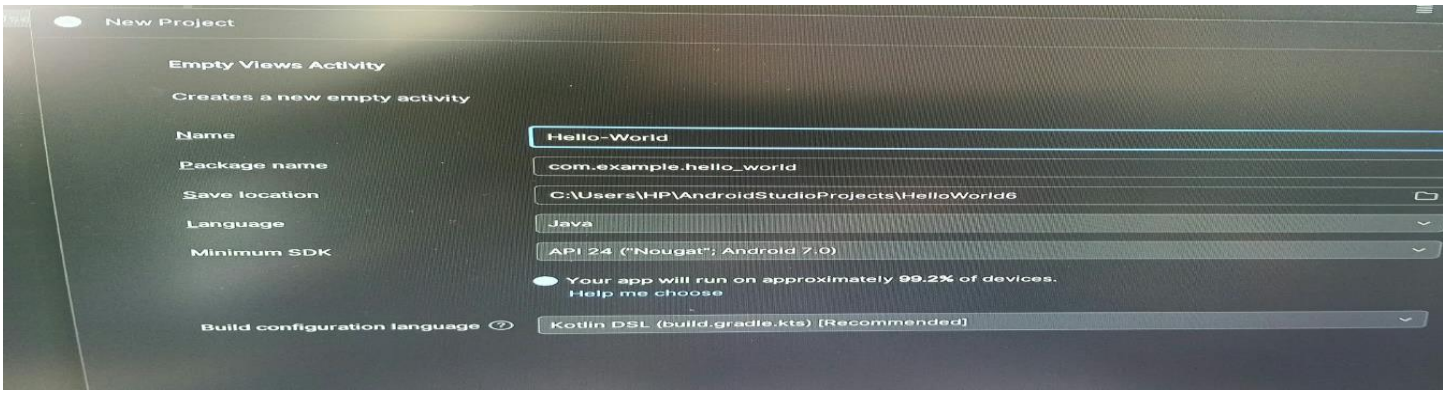

Step 3: Select Template.

Choose: Empty Views Activity  Click: Next.




Step 4: Configure Project

Fill the following details:



**III.MAINACTIVITY.JAVA :**

```
package com.example.helloworld;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
(v, insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });
    }
}
```

**IV. DESIGING USER INTERFACE:**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="102dp"
        android:layout_height="56dp"
        android:text="Hello World!"
        android:textColor="#03A9F4"
        android:textColorHighlight="#FFEB3B"
        android:textColorLink="#9D0E0E"
        android:textSize="16sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
/androidx.constraintlayout.widget.ConstraintLayout>
```

**V.RUN THE APPLICATION :**

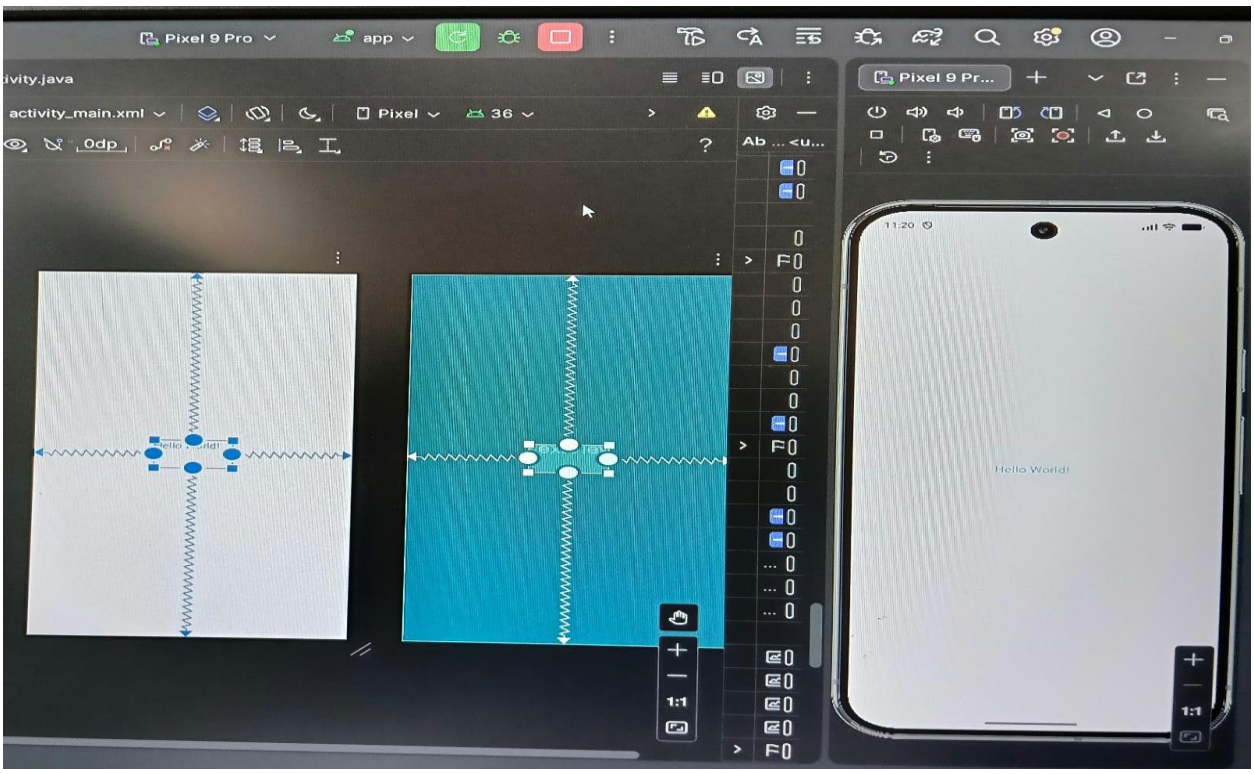
**Step 1 :**

**Click the Run Button :** ➡

**Step 2: Select Emulator Device**

**Step 3:**

**Wait for Gradle build.**



# PRACTICAL 2

DEVELOP AN ANDROID APPLICATION DEMONSTRATING THE USE OF PROGRA-MMING RESOURCES (colors.xml, themes.xml, strings.xml, dimens.xml, DRAWABLE RESOURCES) AND APPLY THEM IN UI COMPONENTS.

## ANDROID BASICS AND RESOURCES

**I.USING STRINGS.XML:**

**Step 1:** Open **strings.xml.**

**Path: res→ values strings.xml**

**Replace code with:**

```
<resources>
<string name="app_name">ResourceDemo</string>
<string name="welcome_text">Welcome to Android Resources</string>
<string name="button_text">Click Here</string>
</resources>
```

**II. USING COLORS.XML:**

**Step 1:** Open **colors.xml**

**Path:**

**res → values → colors.xml**

**Replace code with:**

```
 <resources>
<color name="purple">#6200EE</color>
<color name="white">#FFFFFF</color>
<color name="black">#000000</color>
<color name="buttonColor">#03DAC5</color>
</resources>
colors.xml stores reusable colour values.
```

**III. CREATE DIMENS.XML:**

**Step 1:** Create File.

Right Click values folder from Android Project Pane under res folder**:**

**values → New → Values Resource File**

**File Name: dimens.xml**

**Click: OK**

**Step 2: Add Dimensions.**

```
<resources>
<dimen name="text_size">24sp</dimen>
<dimen name="padding">20dp</dimen>
<dimen name="button_size">18sp</dimen>
</resources>
```

**IV. USING DRAWABLE RESOURCE:**

**Step 1:** Create Drawable File.

**Right Click:**

**Drawable -> New-> Drawable Resource File**

**File Name: button_background.xml**

**Root Element:shape**

**Click: OK**

**Step 2: Add Drawable Code.**

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
<solid android:color="@color/buttonColor"/>
<corners android:radius="12dp"/>
<padding
android:left="10dp"
android:right="10dp"
android:top="10dp"
android:bottom="10dp"/>
</shape>
```

**V. USING themes.xml:**

**Step 1**: Open **themes.xml**

**Path:**

**res→ values → themes.xml**

**Example Theme:**

```
<style name="Theme. Resource Demo"
parent="Theme.Material3.DayNight.NoActionBar">
<item name="colorPrimary">@color/purple</item>
</style>
```

**VI. DESIGN USER INTERFACE:**

**Open: activity_main.xml**

**Path: Res-> layout ->activity_main.xml**

**Replace code with:-**

```
<?xml version="1.0" encoding="utf-8"?>
S<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_gravity="center"
    android:orientation="vertical"
    tools:context=".MainActivity">

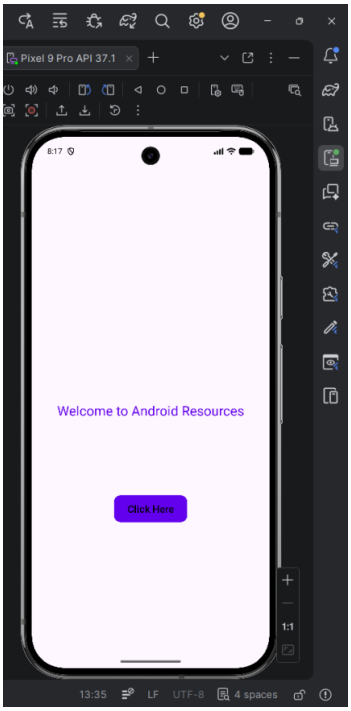
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/welcome_text"
        android:textColor="@color/purple"
        android:textSize="@dimen/text_size"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <Button

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@drawable/button_background"
        android:text="@string/button_text"
        android:textColor="@color/black"
        android:textSize="@dimen/button_size"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.713" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

**VII.RUN THE APPLICATION:**




# PRACTICAL 3

**DEVELOP AN ANDROID APPLICATION DEMONSTRATING COMPLETE ACTIVITY LIFECYCLE METHODS AND ANALYSE EXECUTION USING Logcat.**

## ANDROID BASICS AND RESOURCES

### 1.CREATE NEW ANDROID PROJECT.

Create the Project with **Empty Views Activity** with all details as previously done**.**

Name the Project: **ActivityLifecycleDemo.**

### 2. MODIFY MainActivity.java

**Open:**

**java → MainActivity.java**

Use the Bold Font Code at appropriate place in the file:

```
package com.example.activitylifecycledemo;
import android.util.Log;
```

```
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    String TAG = "ActivityLifecycle";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate() method executed");
    }
    @Override
    protected void onStart() {
        super.onStart();
        Log.d(TAG, "onStart() method executed");
    }
    @Override
    protected void onResume() {
        super.onResume();
        Log.d(TAG, "onResume() method executed");
    }
    @Override
    protected void onPause() {
        super.onPause();
        Log.d(TAG, "onPause() method executed");
    }
    @Override
    protected void onStop() {
        super.onStop();
        Log.d(TAG, "onStop() method executed");
    }
    @Override
    protected void onRestart() {
        super.onRestart();
        Log.d(TAG, "onRestart() method executed");
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        Log.d(TAG, "onDestroy() method executed");
    }
}
```

## III. WHAT IS Logcat?

**Logcat is a debugging tool in Android Studio used to display System logs, Ap Errors and Method execution details.**

## IV. OPEN LOGCAT:

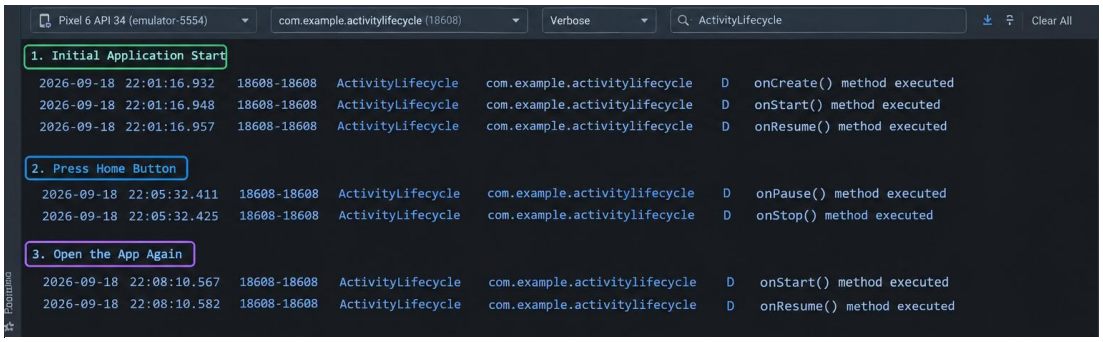
### Method 1:

Click: **View → Tool Windows → Logcat**

## Method 2:

Use **bottom panel: Logcat**

## V. RUN THE APPLICATION.




# PRACTICAL 4

**DEVELOP AN ANDROID APPLICATION TO CREATE AND INTEGRATE A FRAGMENT AND DEMONSTRATE FRAGMENT LIFECYCLE METHODS.**

## FRAGMENTS AND ACTIVITY INTERACTION

### 1. WHAT IS A FRAGMENT?

A Fragment is a reusable portion of user interface inside an Activity

Fragments help developers: Create modular UI, Reuse components and Support table and multiple screen sizes A Fragment always exists inside an Activity.

**Step 1: Create New Android Project.**

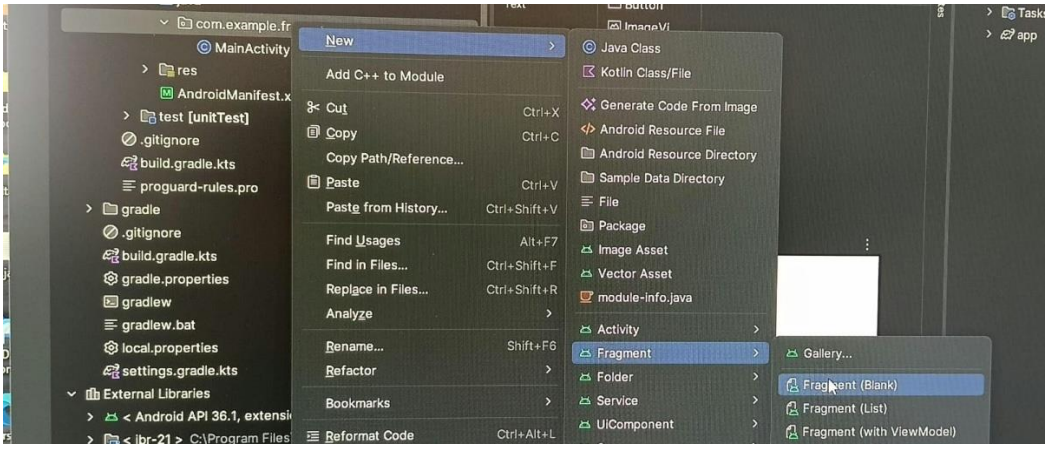
Create the Project with **Empty Views Activity** with all details as previously done

Name the Project: **FragmentLifecycleDemo.**

**Step 2: Create Fragment Class.**

Right Click:

**java package → New → Fragment → Fragment (Blank)**




**Step 3: Give Fragment Details as follows**.

Fragment Name: **DemoFragment**

Fragment Layout Name: **fragment_demo**

Click: **Finish**

**Step 4: Modify DemoFragment.java**.

Open: Use highlighted code at appropriate places in existing code:

```
package com.example.fragmentlifecycledemo;
import android.content.Context;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class DemoFragment extends Fragment {
String TAG = "FragmentLifecycle";
@Override
public void onAttach(@NonNull Context context) {
super.onAttach(context);
Log.d(TAG, "onAttach() executed");
}
@Override
public void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
Log.d(TAG, "onCreate() executed");
}
@Override
public View onCreateView (LayoutInflater inflater, ViewGroup container,
Bundle savedInstanceState) {
Log.d(TAG, "onCreateView() executed");
return inflater.inflate(R.layout.fragment_demo, container, false);
```

```
}
@Override
public void onViewCreated(@NonNull View view,
@Nullable Bundle savedInstanceState) {
super.onViewCreated(view, savedInstanceState);

Log.d(TAG, "onViewCreated() executed");
}

@Override
public void onStart() {
super.onStart();
Log.d(TAG, "onStart() executed");
}

@Override
public void onResume() {
super.onResume();
Log.d(TAG, "onResume() executed");
}

@Override
public void onPause() {
super.onPause();
Log.d(TAG, "onPause() executed");
}

@Override
public void onStop() {
super.onStop();
Log.d(TAG, "onStop() executed");
}

@Override
public void onDestroyView() {
super.onDestroyView();
Log.d(TAG, "onDestroyView() executed");
}

@Override
public void onDestroy() {
super.onDestroy();
Log.d(TAG, "onDestroy() executed");
}

@Override
public void onDetach() {
super.onDetach();
Log.d(TAG, "onDetach() executed");
}}
```

## Step 5: Integrate Fragment in Activity.

**Open: activity_main.xml**

Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/fragment_container"
android:layout_width="match_parent"
android:layout_height="match_parent"/>
```

## Step 6: Modify MainActivity.java.

Replace code with:

```
package com.example.fragmentlifecycledemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.FragmentTransaction;
public class MainActivity extends AppCompatActivity {

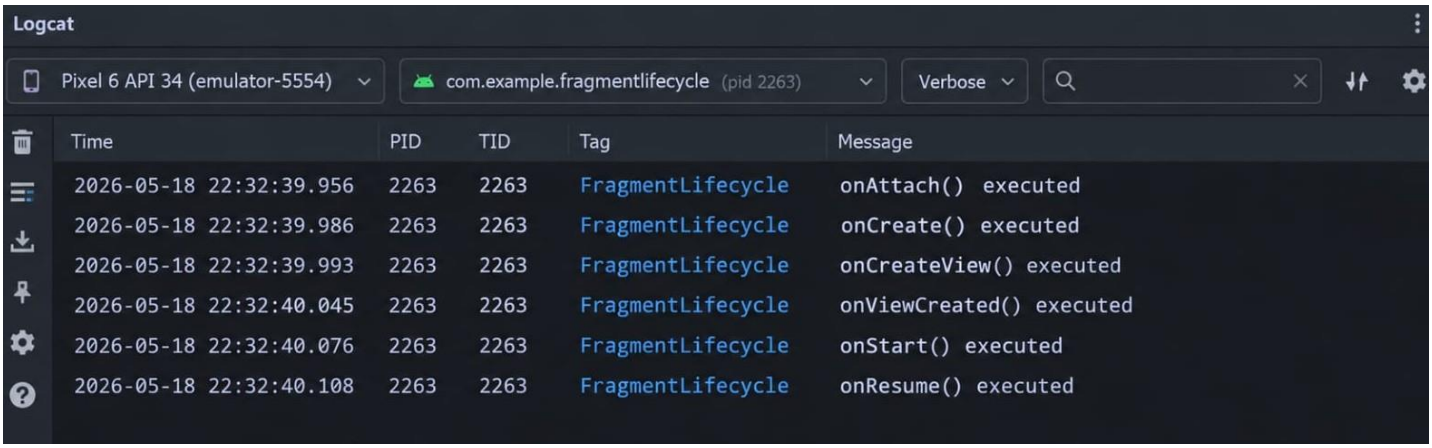
@Override
protected void onCreate(Bundle savedInstanceState) {

super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

DemoFragment fragment = new DemoFragment();

FragmentTransaction transaction =
getSupportFragmentManager().beginTransaction();
transaction.replace(R.id.fragment_container, fragment);
transaction.commit();
}
}
```

**RUN THE APPLICATION:**



# PRACTICAL- 7

## Develop an android application using LinearLayout to design a structured UI

## BASIC LAYOUT MANAGEMENT

**Step 1**: Create new android project.

Name the project **: LinearLayout Attributes.**

**Step 2:** Design user interface using vertical and horizontal LinearLayout.

Open:-

Res -> layout -> activity_main.xml

Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <Button
            android:id="@+id/button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@id/textView6"
            android:layout_alignParentRight="true"
            android:layout_centerHorizontal="true"
            android:layout_marginTop="7dp"
            android:layout_marginRight="155dp"
            android:text="Register"
            android:textSize="24sp" />

        <EditText
            android:id="@+id/editTextText5"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@id/textView5"
            android:layout_alignParentRight="true"
            android:layout_marginTop="7dp"
            android:layout_marginRight="43dp"
            android:ems="10"
               android:hint="address"
            android:inputType="text" />

        <TextView
```

```
        android:id="@+id/textView6"
        android:layout_width="wrap_content"
        android:layout height="wrap content"
        android:layout below="@id/textView5"
        android:layout_marginTop="13dp"
        android:text="Address"
        android:textSize="24sp" />

    <EditText
        android:id="@+id/editTextText3"
        android:layout width="wrap content"
        android:layout_height="wrap_content"
        android:layout below="@id/textView4"
        android:layout_alignParentRight="true"
        android:layout marginTop="8dp"
        android:layout marginRight="52dp"
        android:ems="10"
        android:hint="email id"
        android:inputType="text" />

    <TextView
        android:id="@+id/textView5"
        android:layout width="wrap content"
        android:layout_height="wrap_content"
        android:layout below="@id/textView4"
        android:layout marginTop="15dp"
        android:text="Email Id"
        android:textSize="24sp" />

    <EditText
        android:id="@+id/editTextText2"
        android:layout width="wrap content"
        android:layout height="wrap content"
        android:layout_below="@id/textView3"
        android:layout alignParentRight="true"
        android:layout_marginTop="10dp"
        android:layout marginRight="27dp"
        android:ems="10"
        android:hint="number"
        android:inputType="text" />

    <TextView
        android:id="@+id/textView4"
        android:layout width="wrap content"
        android:layout height="wrap content"
        android:layout_below="@id/textView3"
        android:layout marginTop="12dp"
        android:text="Mobile no."
        android:textSize="24sp" />

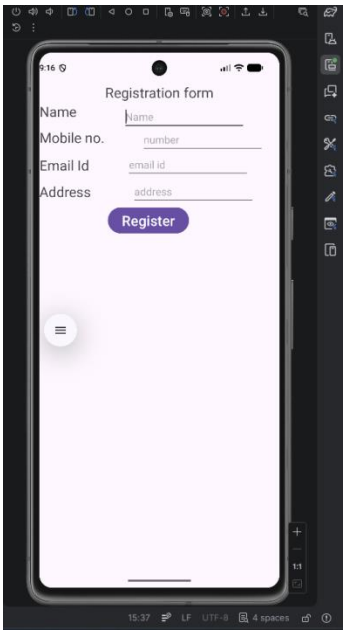
    <EditText
        android:id="@+id/editTextText"
        android:layout_width="wrap_content"
        android:layout height="wrap content"
        android:layout below="@id/textView"
        android:layout_alignRight="@id/textView"
        android:layout alignParentRight="false"
        android:layout_marginTop="3dp"
        android:layout marginRight="-55dp"
        android:ems="10"
        android:hint="Name"
        android:inputType="text" />

    <TextView
        android:id="@+id/textView3"
        android:layout width="wrap content"
        android:layout height="wrap content"
        android:layout_below="@id/textView"
        android:text="Name "
        android:textSize="24sp" />
```

```
        <TextView
            android:id="@+id/textView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_centerHorizontal="true"
            android:text="Registration form"
            android:textColorHighlight="@color/maroon"
            android:textColorHint="@color/maroon"
            android:textColorLink="@color/maroon"
            android:textSize="24sp" />

    </RelativeLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

**Step 3:- Run the application:-**




# PRACTICAL 8

Develop an android application using RelativeLayout to align UI components dynamically.

## BASIC LAYOUT MANAGEMENT

**Step 1**: Create new android project.

Name the project: RelativeLayoutDemo.

**Step 2**: Design user interface using RelativeLayout.

Open:

**res->layout->activity_main.xml**

Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
       <RelativeLayout
        android:layout width="match parent"
        android:layout_height="match_parent">

        <Button
            android:id="@+id/button"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout below="@id/textView6"
            android:layout_alignParentRight="true"
            android:layout_centerHorizontal="true"
            android:layout_marginTop="7dp"
            android:layout_marginRight="155dp"
            android:text="Register"
            android:textSize="24sp" />

        <EditText
            android:id="@+id/editTextText5"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout below="@id/textView5"
            android:layout_alignParentRight="true"
            android:layout_marginTop="7dp"
            android:layout marginRight="43dp"
            android:ems="10"
            android:hint="address"
            android:inputType="text" />

        <TextView
            android:id="@+id/textView6"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout below="@id/textView5"
            android:layout_marginTop="13dp"
            android:text="Address"
            android:textSize="24sp" />

        <EditText
            android:id="@+id/editTextText3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_below="@id/textView4"
            android:layout alignParentRight="true"
            android:layout_marginTop="8dp"
            android:layout marginRight="52dp"
            android:ems="10"
            android:hint="email id"
            android:inputType="text" />

        <TextView
```

```
            android:id="@+id/textView5"
            android:layout_width="wrap_content"
            android:layout height="wrap content"
            android:layout below="@id/textView4"
            android:layout_marginTop="15dp"
            android:text="Email Id"
            android:textSize="24sp" />

        <EditText
            android:id="@+id/editTextText2"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout below="@id/textView3"
            android:layout_alignParentRight="true"
            android:layout marginTop="10dp"
            android:layout marginRight="27dp"
            android:ems="10"
            android:hint="number"
            android:inputType="text" />

        <TextView
            android:id="@+id/textView4"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout below="@id/textView3"
            android:layout marginTop="12dp"
            android:text="Mobile no."
            android:textSize="24sp" />

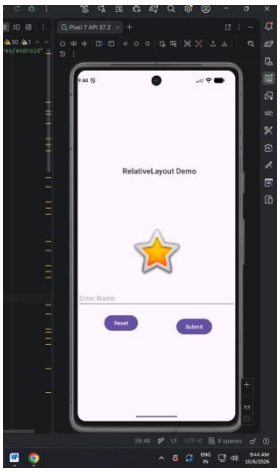
        <EditText
            android:id="@+id/editTextText"
            android:layout width="wrap content"
            android:layout height="wrap content"
            android:layout_below="@id/textView"
            android:layout alignRight="@id/textView"
            android:layout_alignParentRight="false"
            android:layout marginTop="3dp"
            android:layout marginRight="-55dp"
            android:ems="10"
            android:hint="Name"
            android:inputType="text" />

        <TextView
            android:id="@+id/textView3"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:layout below="@id/textView"
            android:text="Name "
            android:textSize="24sp" />

        <TextView
            android:id="@+id/textView"
            android:layout_width="wrap_content"
            android:layout height="wrap content"
            android:layout centerHorizontal="true"
            android:text="Registration form"
            android:textColorHighlight="@color/maroon"
            android:textColorHint="@color/maroon"
            android:textColorLink="@color/maroon"
            android:textSize="24sp" />

    </RelativeLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

**Step 3:RUN the application:**




# PRACTICAL- 9

## Develop an android application using TableLayout to design a Structured login/ registration from

## Step 1:create new android project

Create the project with **Empty Views Activity** with all detail as previously done.

Name the Project**: TableLayoutDemo**

## Step 2:

## Open: res->layout->activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp"
    android:stretchColumns="1">


    <TableRow>
```

```
        <TextView
            android:layout width="match parent"
            android:layout height="match parent"
            android:text="Registration From"
            android:textSize="26sp"
            android:textStyle="bold"
            android:paddingBottom="20dp"/>
    </TableRow>

    <TableRow>
    <TextView
        android:layout width="match parent"
        android:layout_height="match_parent"
        android:text="Name"
        android:textSize="18sp"/>


    <EditText
        android:id="@+id/etName"
        android:layout width="match parent"
          android:layout_height="match_parent"
          android:hint="Enter Name"/>

</TableRow>

<TableRow>
    <TextView
        android:layout width="wrap content"
        android:layout_height="wrap_content"
        android:text="Email:"
        android:textSize="18sp"/>

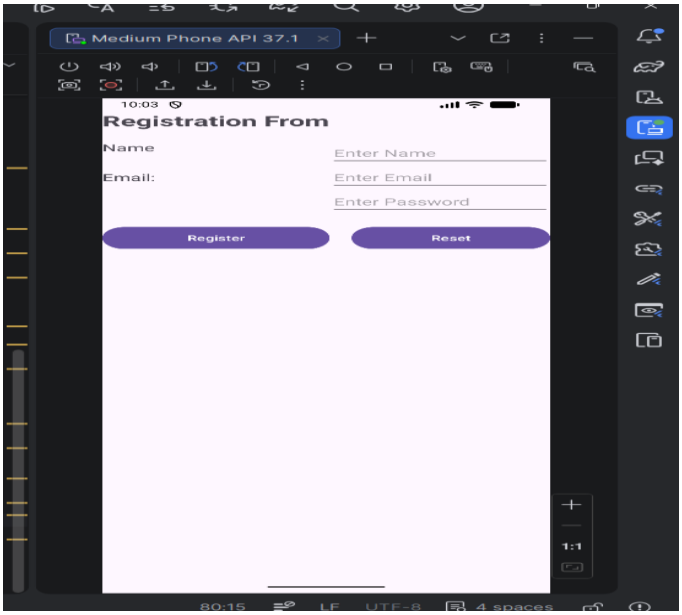
    <EditText
        android:id="@+id/etEmail"
        android:layout width="match parent"
        android:layout_height="wrap_content"
        android:hint="Enter Email"/>
</TableRow>

    <TableRow>
        <TextView
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:textSize="18sp"/>

        <EditText
            android:id="@+id/etPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter Password"
            android:inputType="textPassword"/>
    </TableRow>
    <TableRow
        android:paddingTop="20dp">
        <Button
            android:id="@+id/btnRegister"
            android:layout width="wrap content"
            android:layout_height="wrap_content"
            android:text="Register"/>

        <Button
            android:id="@+id/btnReset"
            android:layout_width="wrap_content"
            android:layout height="wrap content"
            android:text="Reset"
            android:layout_marginStart="20dp"/>
    </TableRow>
</TableLayout>
```

**Step 3: RUN the application**




## PRACTICAL- 10

**Develop an android application using SCROLLVIEW to display long Scrollable content**

**Step 1: create new android project**

Create the project with **Empty view activity** with all details as previously done.

Name the project: ***ScrollViewDemo***

Step 2: **Design Scrollable User Interface**

**Open: Res->layout->activity_main.xml**

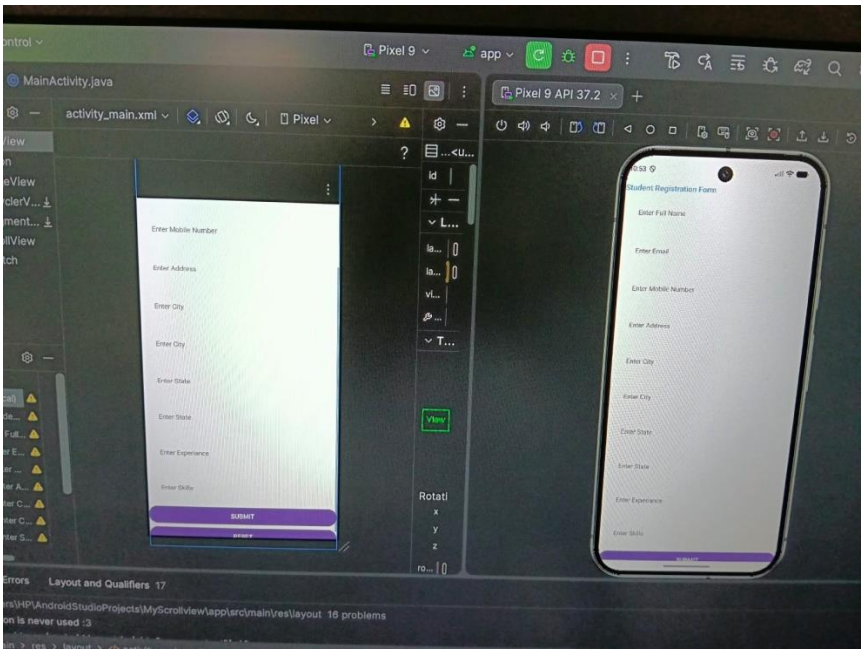
```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
χαλης:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:id="@+id/textView11"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Student Registration Form"
        android:textColor="#2196F3"
        android:textColorHighlight="#FF9800*
android:textColorLink="#A41414"
    android:textSize="16sp"
    android:textStyle="bold" />
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_marginBottom="10dp"
        android:layout_weight="1"
        android:text="Enter Full Name" />
    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter Email"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textView3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter Mobile Number"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textView4"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter Address"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textView5"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter City"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textViewó"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter City"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textView7"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter State"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textView8"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter State"
        android:layout_marginBottom="10dp"/>

    <TextView
        android:id="@+id/textView9"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter Experiance"
        android:layout_marginBottom="10dp"/>
    <TextView
        android:id="@+id/textView10"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="30dp"
        android:layout_weight="1"
        android:text="Enter Skills"
        android:layout_marginBottom="10dp"/>
    <Button
        android:id="@+id/button"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="SUBMIT" />
    <Button
        android:id="@+id/button2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="RESET" />
</LinearLayout>
</ScrollView>
```

**Step 3: RUN the application**




# PRACTICAL 12

## Develop an android application using FrameLayout to overlay UI elements.

## ADAVANCED LAYOUT AND VIEW HANDLING

## Step 1:Create new android project.

Name the project: **FrameLayoutDemo.**

**S**tep 2: Design user interface using FrameLayout.

## Open:res->layout->activity_main.xml

## Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

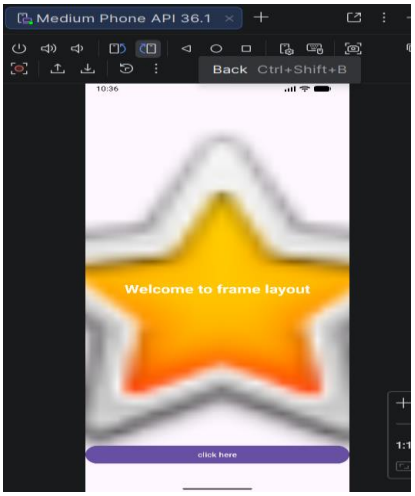

        <ImageView
            android:id="@+id/imageView"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            app:srcCompat="@android:drawable/btn_star_big_on"
            android:scaleType="centerCrop"/>

        <TextView
            android:id="@+id/textView"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Welcome to frame layout"
            android:textSize="26sp"
            android:textStyle="bold"
            android:textColor="#FFFFFF"
            android:layout_gravity="center"/>

        <Button
            android:id="@+id/button"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_gravity="bottom"
            android:layout_marginBottom="40dp"
            android:text="click here" />


    </FrameLayout>
```

## Step 3: RUN the application



# PRACTICAL 13

## Develop an android application using ListView with array adapter and implement item click handling.

## ADAPTER BASED VIEWS

**Step 1:** Create new android project.

Name the project: **ListViewDemo.**

**Step 2: Design user interface.**

**Open: res->layout->activity_main.xml**

**Replace code with:**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"

    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="10dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Programming Languages"
        android:textSize="24sp"
        android:textStyle="bold"
        android:paddingBottom="10dp" />
    <ListView
       android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</LinearLayout>
```

**Step 3: Modify MainActivity.java**

**Open:**

**MainActivity.java**

Replace code with:

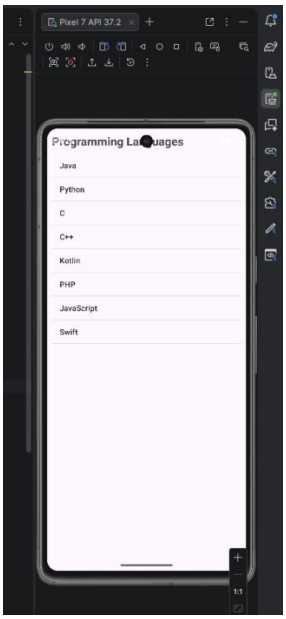
```
package com.example.listviewdemo;

import android.os.Bundle;
import android.view.View;
import android.widget.Adapter;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {
    ListView listView;
    String[] languages = {
            "Java",
            "Python",
            "C",
            "C++",
            "Kotlin",
            "PHP",
            "JavaScript",
            "Swift",
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity main);
        listView = findViewById(R.id.listView);
        //Create ArrayAdapter
        ArrayAdapter<String> adapter =
                new ArrayAdapter<>(
                        this,
                        android.R.layout.simple_list_item_1,
                        languages
                );
        //Connect Adapter with ListView
        listView.setAdapter(adapter);
        //Item Click Handling
        listView.setOnItemClickListener(
                new AdapterView.OnItemClickListener() {
                    @Override
                    public void onItemClick(AdapterView<?> parent,
                                            View view,
                                            int position,
                                            long id) {
                        String item = languages[position];
                        Toast.makeText(
                                MainActivity.this,
                                "Selected:" + item,
                                Toast.LENGTH SHORT
                        ).show();
                    }
                });
    }
}
```

**Step 4: RUN the application:**




# PRACTICAL 14

**Develop an android application using GRIDVIEW with custom adapter and image selection handling.**

## ADAPTER BASED VIEWS

Step 1: Create new android practical.

Name the project: **GridViewDemo.**

Step 2: Add Some images to Drawable folder.

res->drawable

Examples image names

1.image1.jpg

2.image2.jpg

3.image3.jpg

4.image4.jpg

## Step 3:Design main layout.

## Open:res->layout->activity_main_xml

## Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="10dp">


    <GridView
        android:id="@+id/gridView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:horizontalSpacing="10dp"
        android:numColumns="2"
        android:stretchMode="columnWidth"
        android:verticalSpacing="10dp"/>




</LinearLayout>
```

## Step 4: Create custom grid Item layout.

## res->layout->new->XML->new layout XML file ->grid_item

## Add following code:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="10dp">
    <ImageView
        android:id="@+id/imageView"
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:scaleType="centerCrop" />
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:textSize="18sp"
        android:paddingTop="5dp"/>


</LinearLayout>
```

**Step 5: Create Custom adapter class.**

Right click:

**Java package->New ->Java Class**

Class name: **CustomAdapter.**

**Add code:**

```
package com.example.gridviewdemo;
import android.content.Context;
import android.view.LayoutInflater;

import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.ImageView;
import android.widget.TextView;
public class CustomAdapter extends BaseAdapter {
    Context context;
    String[] imageNames;
    int[] images;
    LayoutInflater inflater;

    public CustomAdapter(Context context,
                         String[] imageNames,
                         int[] images) {
        this.context = context;
        this.imageNames = imageNames;
        this.images = images;
        inflater = LayoutInflater.from(context);
    }
        @Override
        public int getCount ()
        {
            return imageNames.length;
        }
        @Override
        public Object getItem ( int position)
        {
            return imageNames[position];
        }
        @Override
        public long getItemId ( int position)
        {
            return position;
        }
        @Override
        public View getView ( int position, View convertView, ViewGroup parent)
        {
            convertView = inflater.inflate(R.layout.grid_iitem, parent, false);
            ImageView imageView = convertView.findViewById(R.id.imageView);
            TextView textView = convertView.findViewById(R.id.textView);
```

```
            imageView.setImageResource(images[position]);
            textView.setText(imageNames[position]);
            return convertView;
        }

    }
```

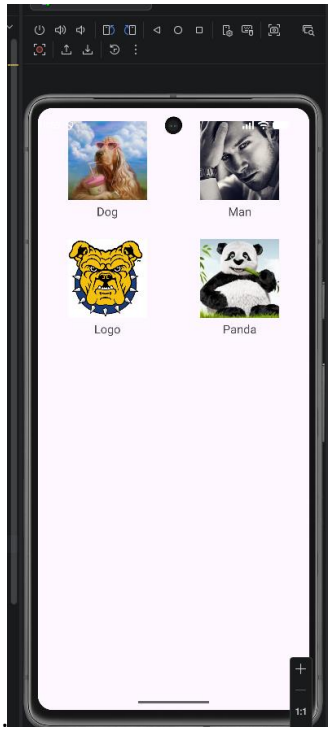
## Step 6: Modify MainActivity.java.

## Replace code with:

```
package com.example.gridviewdemo;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.GridView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.example.gridviewdemo.CustomAdapter;
public class MainActivity extends AppCompatActivity {
    GridView gridView;
    String[] imageNames={
            "Dog",
            "Man",
            "Logo",
            "Panda"};
    int[]images={
            R.drawable.dog,
            R.drawable.man,
            R.drawable.ncat_logo,
            R.drawable.panda
    };
 @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity main);
        gridView=findViewById(R.id.gridView);
        CustomAdapter adapter=
                new CustomAdapter(this, imageNames, images);
        gridView.setAdapter(adapter);
        /*  gridView.setOnItemClickListener(
                new AdapterView.OnItemClickListener() {
                    @Override
                    public void onItemClick(AdapterView<?> parent, android.view.View view, int
position, long id) {
                        Toast.makeText(
                                MainActivity.this,
                                        "Selected:"+
                                        imageNames[position],
                                Toast.LENGTH SHORT
                        ).show();
   }} );*/

    }
     }
```

## Step 7: RUN the Application




# MODULE 2

# PRACTICAL 1

## Develop An Android Application Implementing AppBar (Toolbar) And Options Menu.

**STEP 1: : Create New Android Project.**

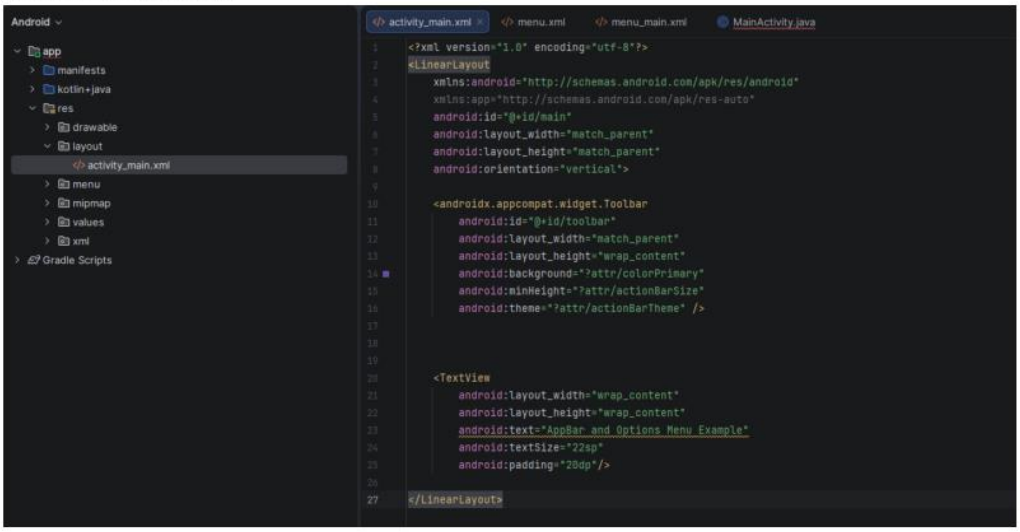
Create the project with **Empty views Activity** with all details as previously done.
Name the Project: **Toolbar MenuDemo.**

**Step 2: Design User Interface.**

**Open:**

**Res ⟶ Layout ⟶ activity_main.xml**

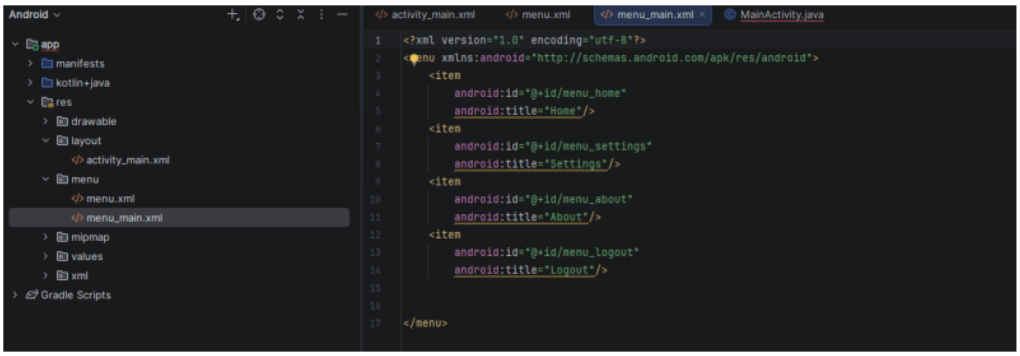
Replace code with:




**Step 3: Create Menu XML File.**

**Menu ⟶ New ⟶ Menu Resource File**

Add code:

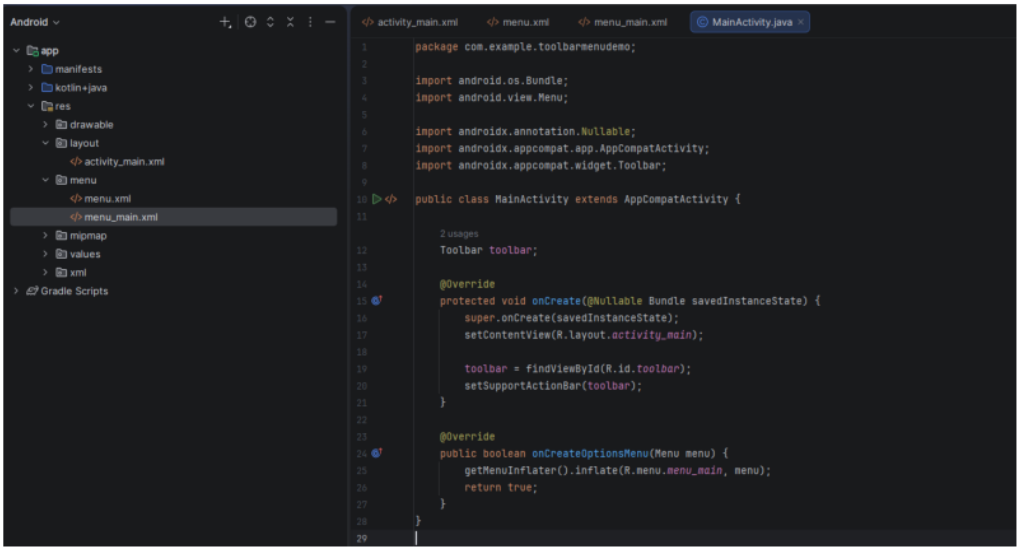



**Step 5: Modify MainActivity.java.**

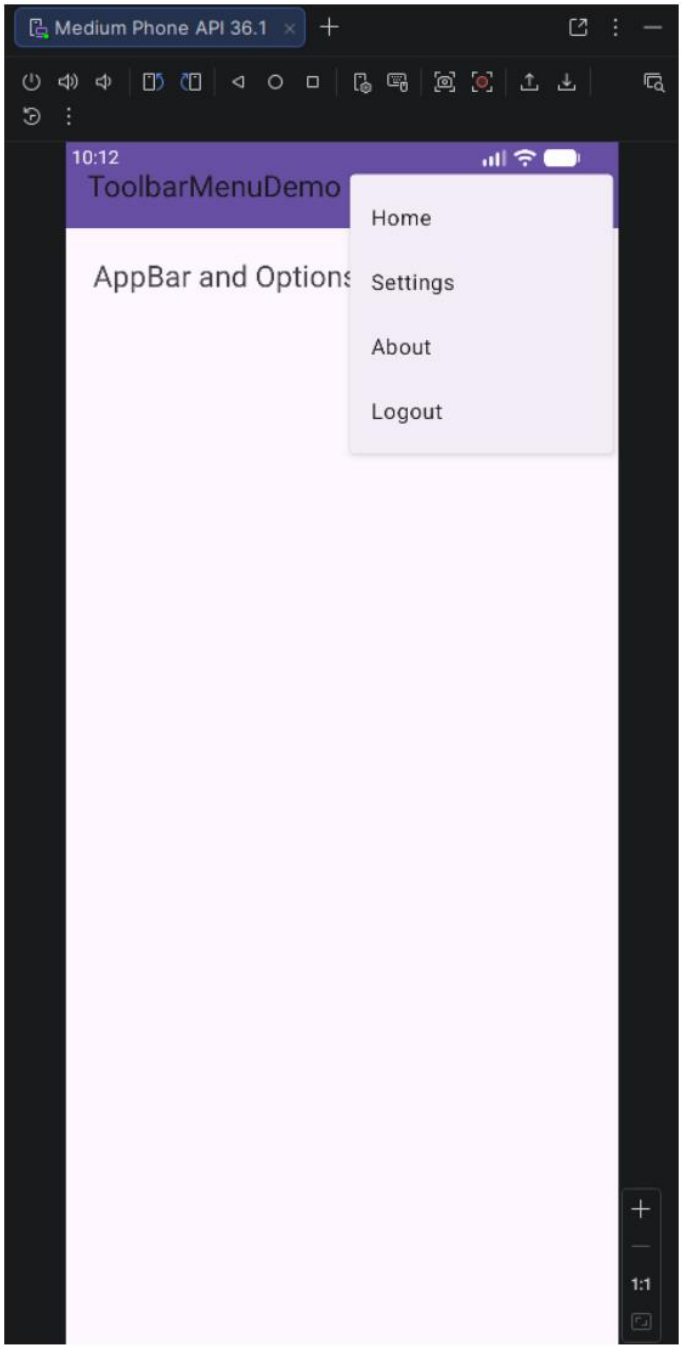
**Open:**

**MainActivity.java**

Replace code with:




**Step 6: Run The Application**


Medium Phone API 36.1
10:12
ToolbarMenuDemo
AppBar and Options
Home
Settings
About
Logout
1:1

# PRACTICAL 2

## Develop An Android Application Demonstrating AlertDialog And Custom DialogFragment.

**STEP 1: : Create New Android Project.**

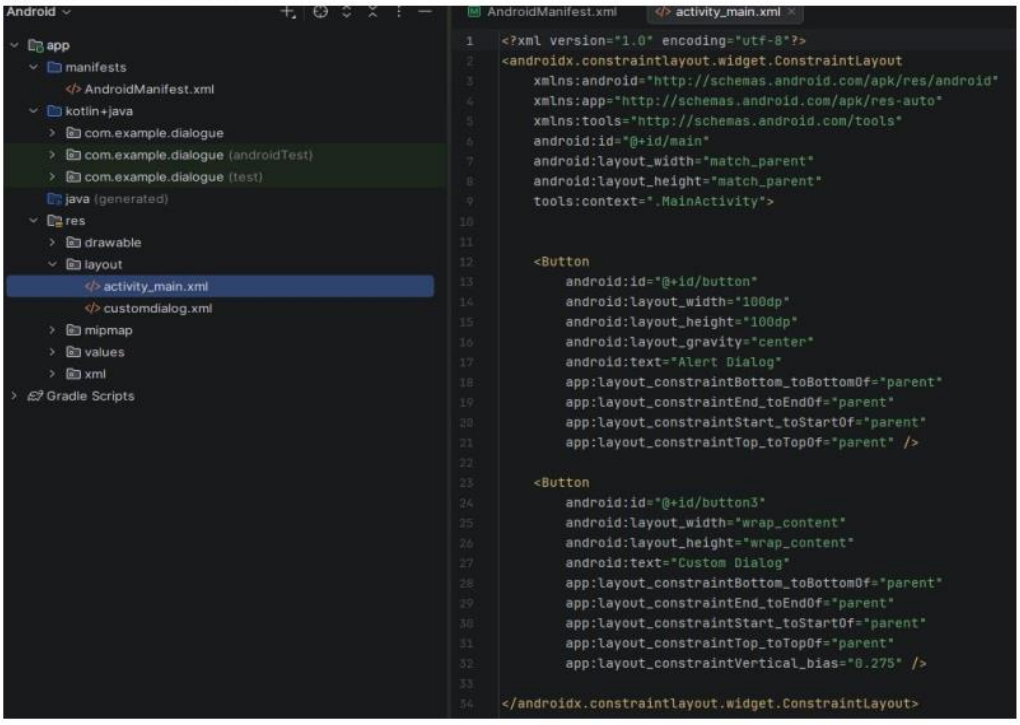
Create the project with **Empty views Activity** with all details as previously done.
Name the Project: **DialogDemo.**

**Step 2: Design Main Layout.**

**Open:**

**Res ⟶ layout ⟶ activity_main.xml**

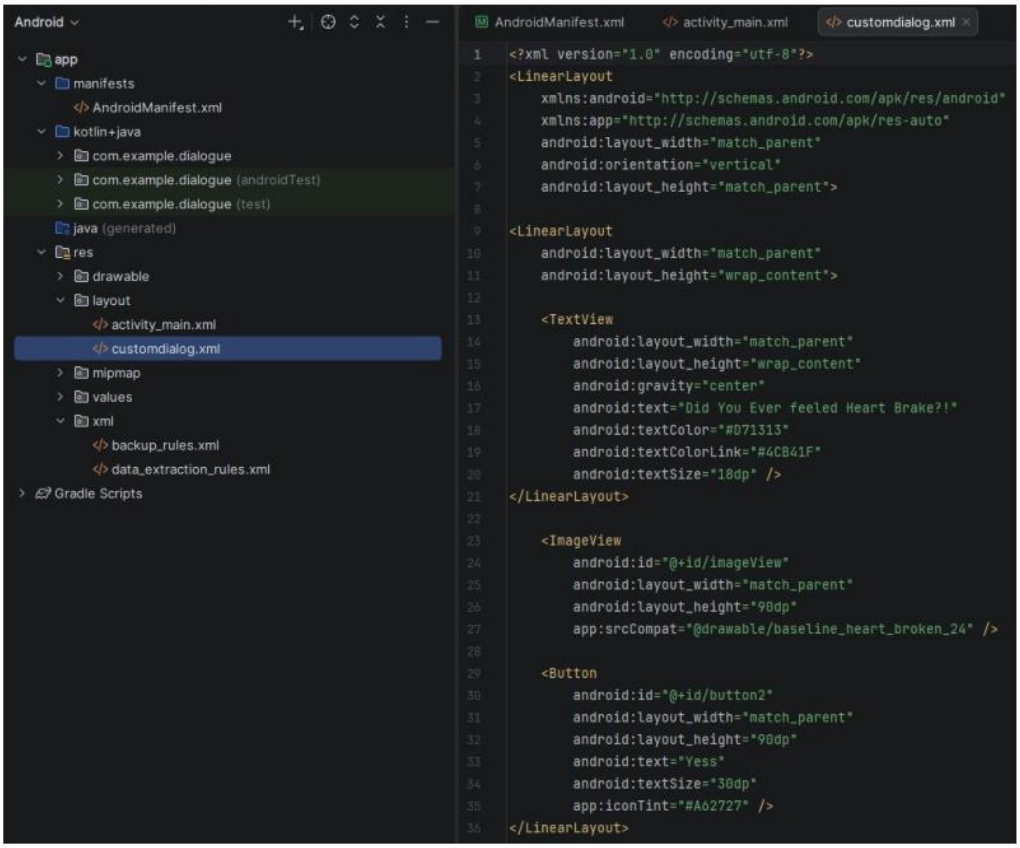
**Replace Code With:**



**Step 3: Create Custom Dialog Layout.**

**Create:**

**res ⟶ layout ⟶ New -> XML -> New Layout XML File ⟶ dialog_custom**




**Step 4: Modify MainActivity.java.**

**Open:**

**MainActivity.java**

Replace code with:

AndroidManifest.xml | activity_main.xml | MainActivity.java | customdialog.xml

```java
package com.example.dialogue;

import ...

public class MainActivity extends AppCompatActivity {
    2 usages
    Button btn,btn1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        btn= findViewById(R.id.button);
        btn1= findViewById(R.id.button3);
        btn1.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Dialog dialog=new Dialog( context: MainActivity.this);
                dialog.setContentView(R.layout.customdialog);
                Button bo=dialog.findViewById(R.id.button2);
                dialog.setCancelable(true);

                bo.setOnClickListener(new View.OnClickListener() {
                    @Override
                    public void onClick(View v) {
                        Toast.makeText( context: MainActivity.this, text: "Don't Worry Comeback Stronger!!):",Toast.LENGTH_SHORT).show();
                        dialog.dismiss();
                    }
                });
                dialog.setTitle("Hello");
                dialog.show();
            }
        });
        btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {...}
        });
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
```

```java
        });
        btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                AlertDialog.Builder alert=new AlertDialog.Builder( context: MainActivity.this);
                alert.setTitle("Warning!");
                alert.setMessage("Do you Still want to continue?");
                alert.setPositiveButton( text: "Yes", new DialogInterface.OnClickListener() {
                    @Override
                    public void onClick(DialogInterface dialog, int which) {
                        Toast.makeText( context: MainActivity.this, text: "You Selected Yes",Toast.LENGTH_SHORT).show();
                    }
                });
                alert.setNegativeButton( text: "No", new DialogInterface.OnClickListener() {
                    @Override
                    public void onClick(DialogInterface dialog, int which) {
                        Toast.makeText( context: MainActivity.this, text: "You Selected No",Toast.LENGTH_SHORT).show();
                    }
                });
                alert.show();
            }
        });
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}
```

**Step 5: Run The Application.**



# PRACTICAL 3

# Develop An Android Application Implementing Contextual Menu And Popup Menus.

## STEP 1: : Create New Android Project.

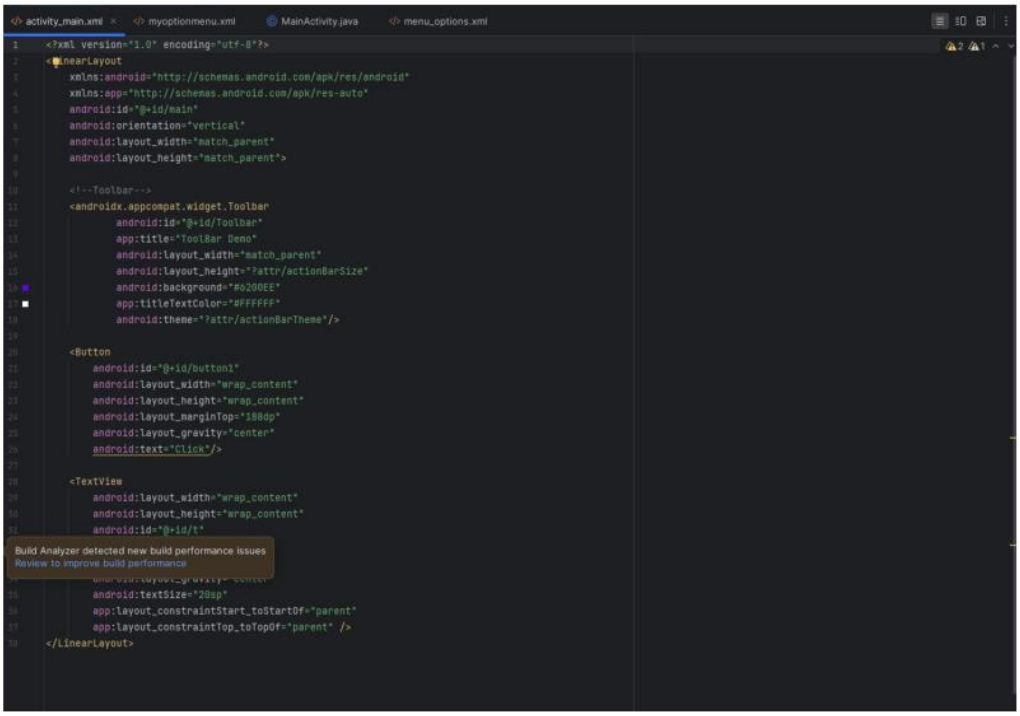
Create the project with **Empty views Activity** with all details as previously done.
Name the Project: **MenuDemo.**

**Step 2: Design User Interface.**

**Open:**

**Activity_main.xml**

Replace code with:



**Step 3: Create Menu resource File.**

**Right Click:**

**res ⟶ New ⟶ Android Resource Directory**

Directory Name: **menu**

Resource Type: **menu**

Click: **OK**

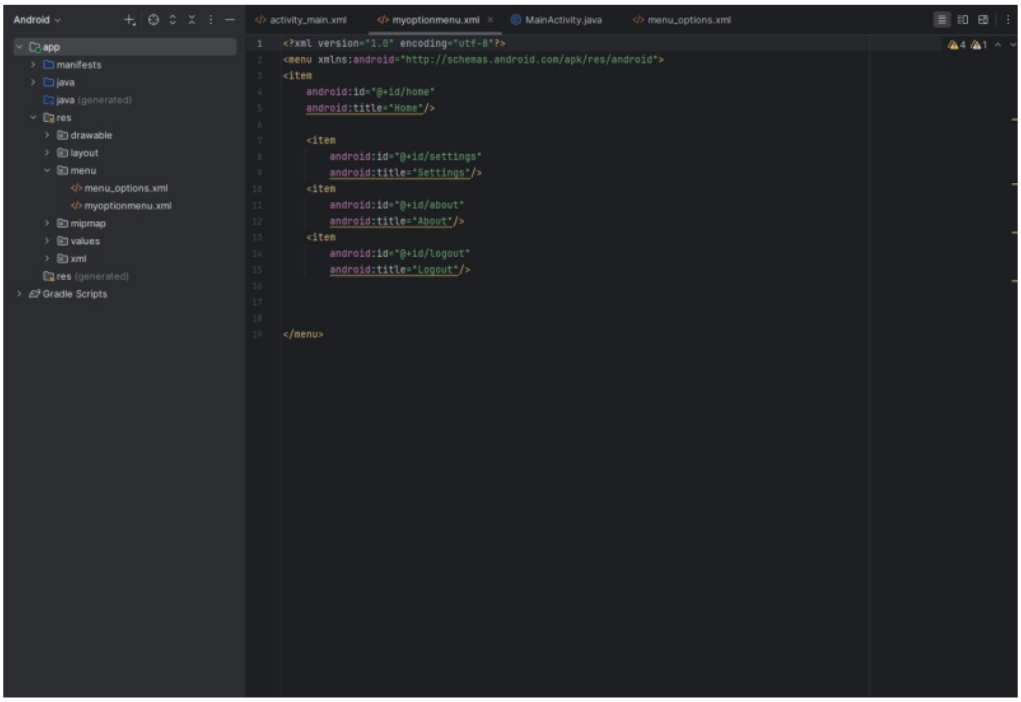
Right Click:

**Menu ⟶ New ⟶ Menu Resource File**

File Name: **menu_Options**

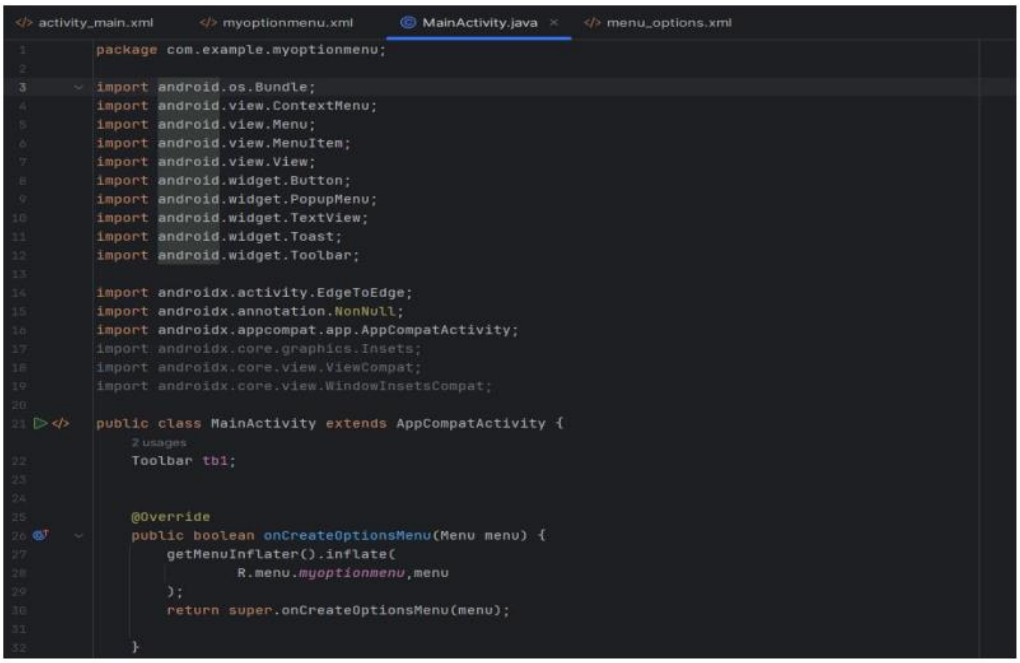
Click: **OK**

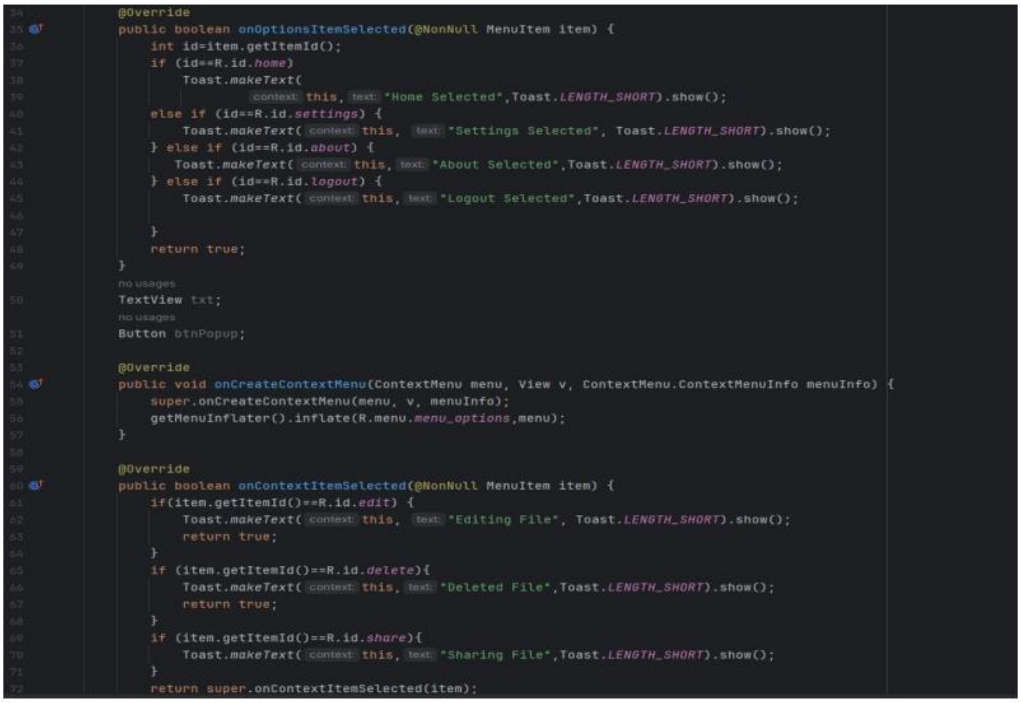
Add Code:




**Step 4: Modify MainActivity.java.**

Replace code with:


activity_main.xml
myoptionmenu.xml
MainActivity.java
menu_options.xml
package com.example.myoptionmenu;
import android.os.Bundle;
import android.view.ContextMenu;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;
import android.widget.Toolbar;
import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
public class MainActivity extends AppCompatActivity {
2 usages
Toolbar tb1;
@Override
public boolean onCreateOptionsMenu(Menu menu) {
getMenuInflater().inflate(
R.menu.myoptionmenu,menu
);
return super.onCreateOptionsMenu(menu);
}


@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
int id=item.getItemId();
if (id==R.id.home)
Toast.makeText(
context: this, text: "Home Selected",Toast.LENGTH_SHORT).show();
else if (id==R.id.settings) {
Toast.makeText( context: this, text: "Settings Selected", Toast.LENGTH_SHORT).show();
} else if (id==R.id.about) {
Toast.makeText( context: this, text: "About Selected",Toast.LENGTH_SHORT).show();
} else if (id==R.id.logout) {
Toast.makeText( context: this, text: "Logout Selected",Toast.LENGTH_SHORT).show();
}
return true;
}
no usages
TextView txt;
no usages
Button btnPopup;
@Override
public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) {
super.onCreateContextMenu(menu, v, menuInfo);
getMenuInflater().inflate(R.menu.menu_options,menu);
}
@Override
public boolean onContextItemSelected(@NonNull MenuItem item) {
if(item.getItemId()==R.id.edit) {
Toast.makeText( context: this, text: "Editing File", Toast.LENGTH_SHORT).show();
return true;
}
if (item.getItemId()==R.id.delete){
Toast.makeText( context: this, text: "Deleted File",Toast.LENGTH_SHORT).show();
return true;
}
if (item.getItemId()==R.id.share){
Toast.makeText( context: this, text: "Sharing File",Toast.LENGTH_SHORT).show();
}
return super.onContextItemSelected(item);

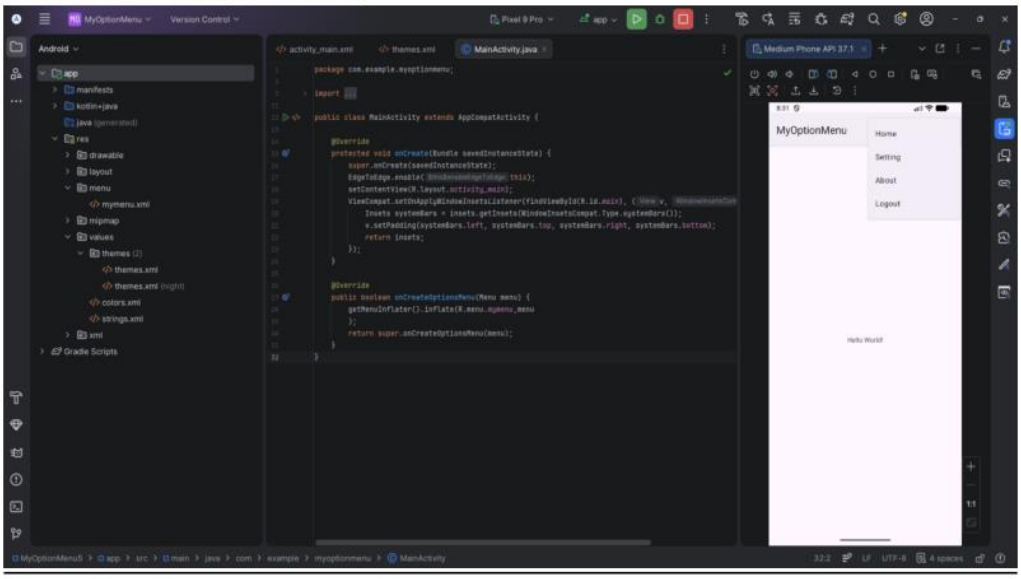
```
    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        tb1 = findViewById(R.id.Toolbar);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        TextView targetTextView = findViewById(R.id.t);
        Button btn = findViewById(R.id.button1);
        registerForContextMenu(tb1);
        btn.setOnClickListener( View v -> {
            PopupMenu popup=new PopupMenu( context: MainActivity.this,btn);
            popup.getMenuInflater().inflate(R.menu.menu_options,popup.getMenu());
            popup.setOnMenuItemClickListener( MenuItem item -> {
                Toast.makeText( context: MainActivity.this,item.getTitle(),Toast.LENGTH_SHORT).show();
                return true;
            });
            popup.show();
        });

    };
}
```

## Step 5: Run The Application.



# PRACTICAL 4

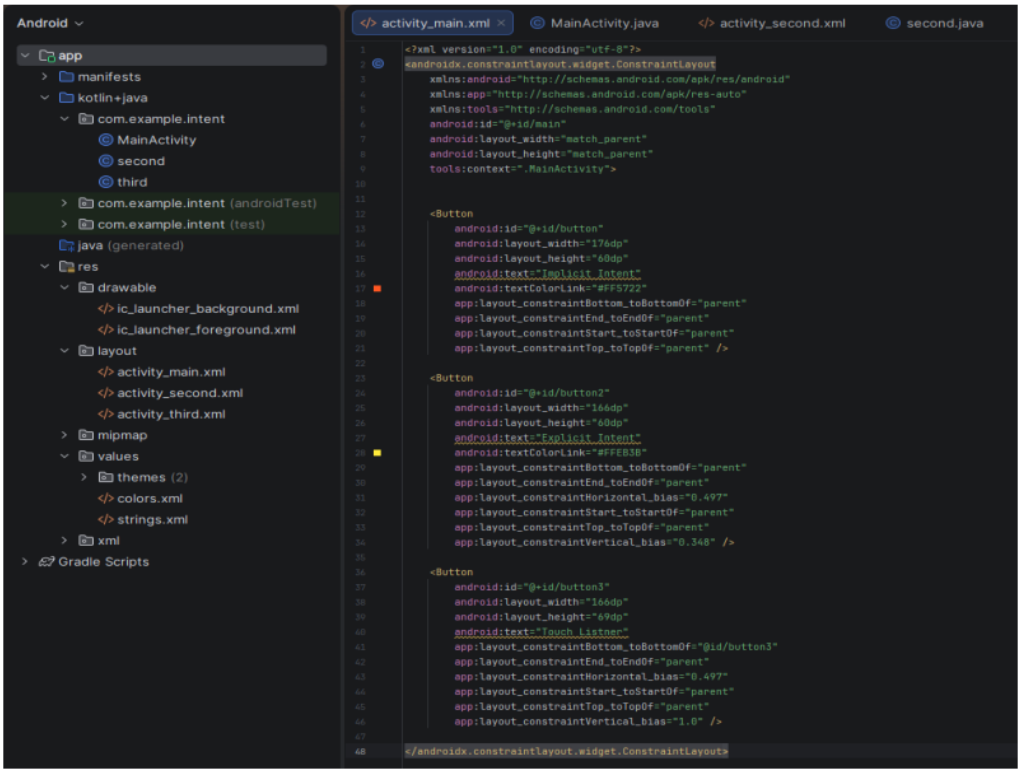
## Develop An Android Application Implementing Explicit And Implicit Intents.

### STEP 1: : Create New Android Project.

Create the project with **Empty views Activity** with all details as previously done. Name the Project: **IntentDemo.**

### Step 2: Create First Activity Layout.

Activity_main.xml



**Step 3: Create Second Activity.**

Right Click:

**Java Package ⟶ New ⟶ Activity ⟶ Empty Views Activity**

Activity Name: **Second Activity Layout.**

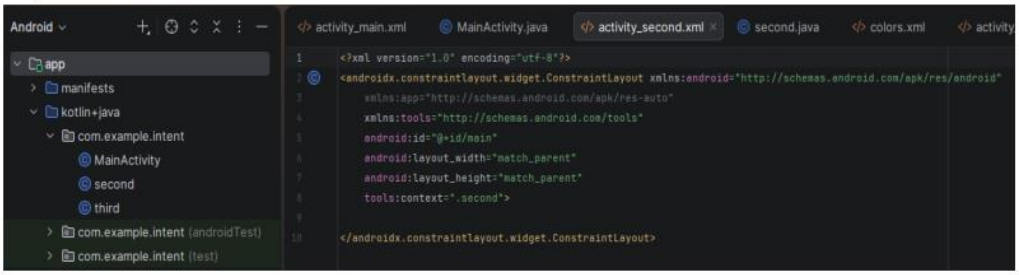
Click: **Finish**

**Step 4: Design Second Activity Layout.**

**Open:**

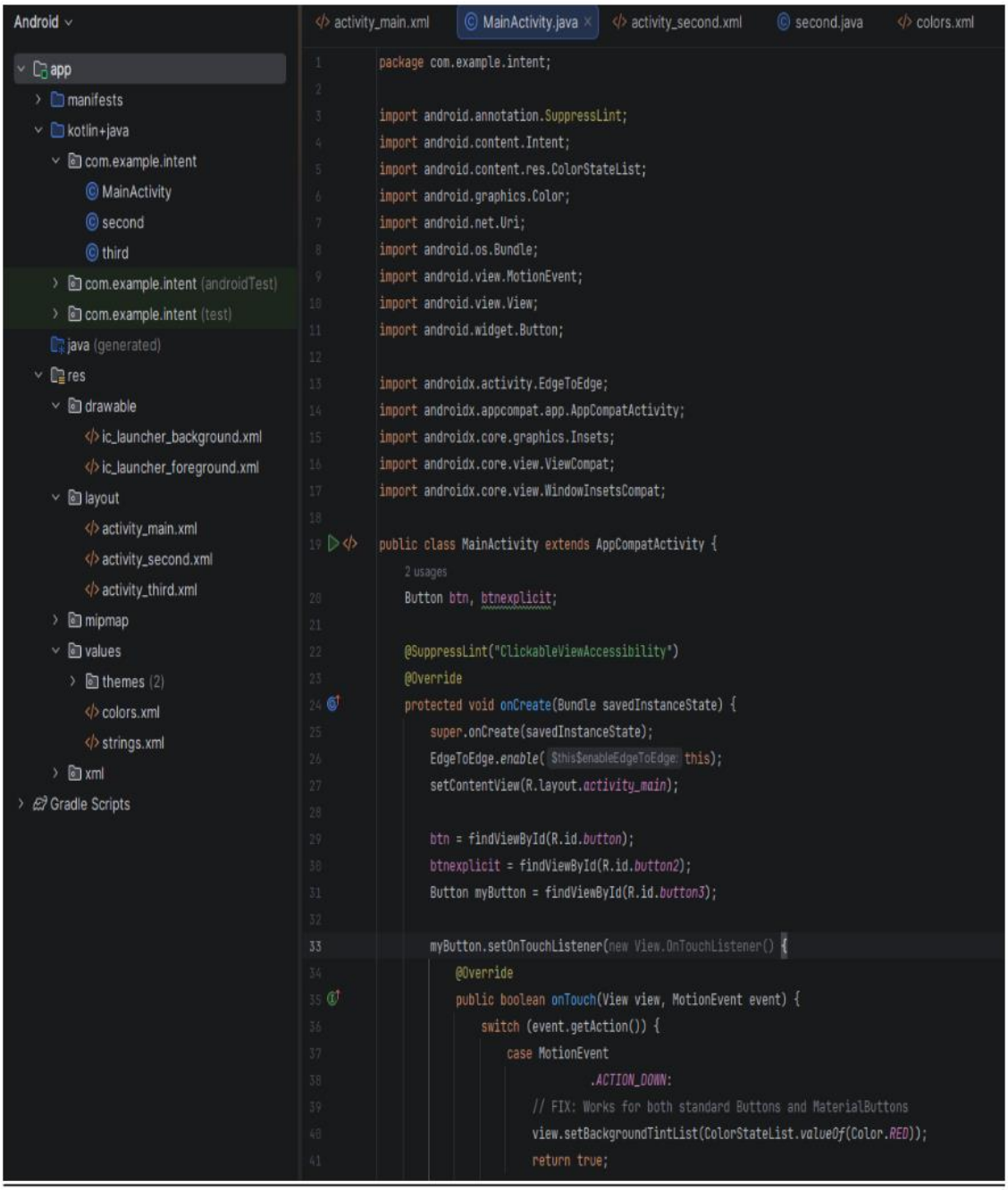
**res ⟶ layout ⟶ activity_second.xml**

Replace code with:




**Step 5: Modify MainActivity.java.**

Replace code with:


Android
app
manifests
kotlin+java
com.example.intent
MainActivity
second
third
com.example.intent (androidTest)
com.example.intent (test)
java (generated)
res
drawable
ic_launcher_background.xml
ic_launcher_foreground.xml
layout
activity_main.xml
activity_second.xml
activity_third.xml
mipmap
values
themes (2)
colors.xml
strings.xml
xml
Gradle Scripts
activity_main.xml
MainActivity.java
activity_second.xml
second.java
colors.xml
package com.example.intent;
import android.annotation.SuppressLint;
import android.content.Intent;
import android.content.res.ColorStateList;
import android.graphics.Color;
import android.net.Uri;
import android.os.Bundle;
import android.view.MotionEvent;
import android.view.View;
import android.widget.Button;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
public class MainActivity extends AppCompatActivity {
2 usages
Button btn, btnexplicit;
@SuppressLint("ClickableViewAccessibility")
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
EdgeToEdge.enable( $this$enableEdgeToEdge: this);
setContentView(R.layout.activity_main);
btn = findViewById(R.id.button);
btnexplicit = findViewById(R.id.button2);
Button myButton = findViewById(R.id.button3);
myButton.setOnTouchListener(new View.OnTouchListener() {
@Override
public boolean onTouch(View view, MotionEvent event) {
switch (event.getAction()) {
case MotionEvent
.ACTION_DOWN:
// FIX: Works for both standard Buttons and MaterialButtons
view.setBackgroundTintList(ColorStateList.valueOf(Color.RED));
return true;

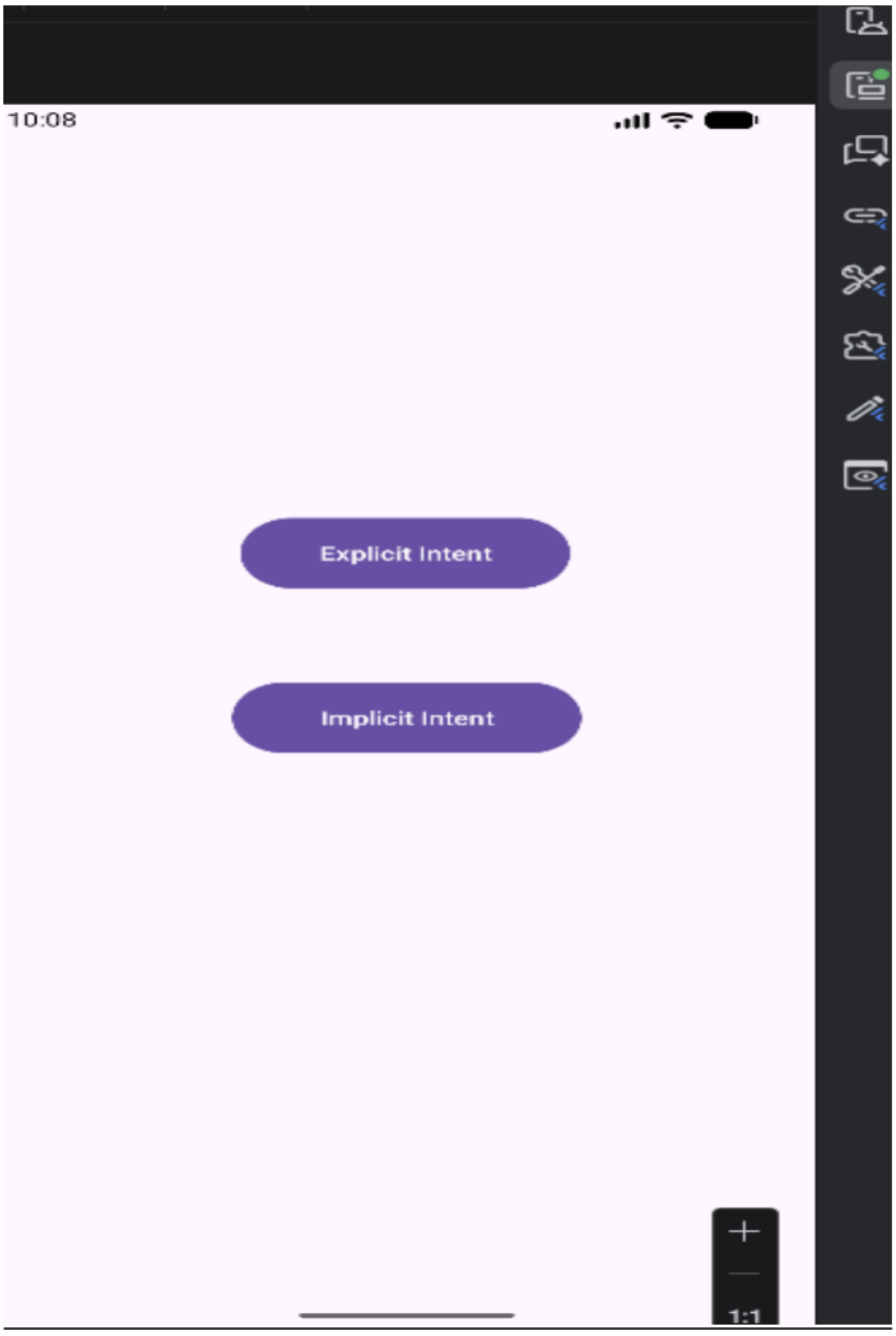
```
                    case MotionEvent.ACTION_MOVE:
                        view.setBackgroundTintList(ColorStateList.valueOf(Color.BLACK));
                        return true;

                    case MotionEvent.ACTION_UP:
                        view.setBackgroundTintList(ColorStateList.valueOf(Color.BLUE));
                        return true;

                    case MotionEvent.ACTION_CANCEL:
                        view.setBackgroundTintList(ColorStateList.valueOf(Color.YELLOW));
                        return true;
                }
                return false;
            }
        });
        btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent i = new Intent(Intent.ACTION_VIEW, Uri.parse( uriString: "https://www.google.com"));
                startActivity(i);
            }
        });
        btnexplicit.setOnLongClickListener(new View.OnLongClickListener() {
            @Override
            public boolean onLongClick(View v) {
                //Assuming "second" is your targeted activity class name
                Intent i = new Intent( packageContext: MainActivity.this, second.class);
                startActivity(i);
                return true;// FIX: Changed from false to true to consume the event
            }
        });

        //FIX: Ensure your activity_main.xml root layout has android:id="@+id/main"
        View mainView = findViewById(R.id.main);
        if (mainView != null) {
            ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v,  WindowInsetsCompat insets) -> {
                Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
                v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
                return insets;
            });
        }
    }
}
```

## Step 6: Run The Application.


10:08
Explicit Intent
Implicit Intent
1:1

# PRACTICAL 5

## Develop An Android Application Demonstrating Event Handling Using Listener (OnClick,OnLongClick,Etc.).

### STEP 1: Create New Android Project.

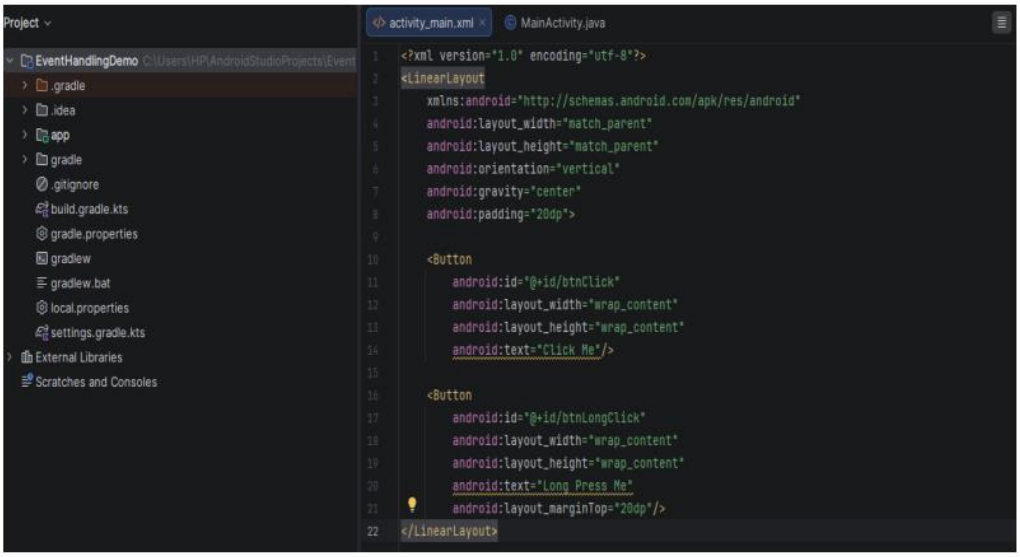
Create the project with **Empty views Activity** with all details as previously done.
Name the Project: **EventHandlingDemo.**

### Step 2: Design User Interface

**Open:**

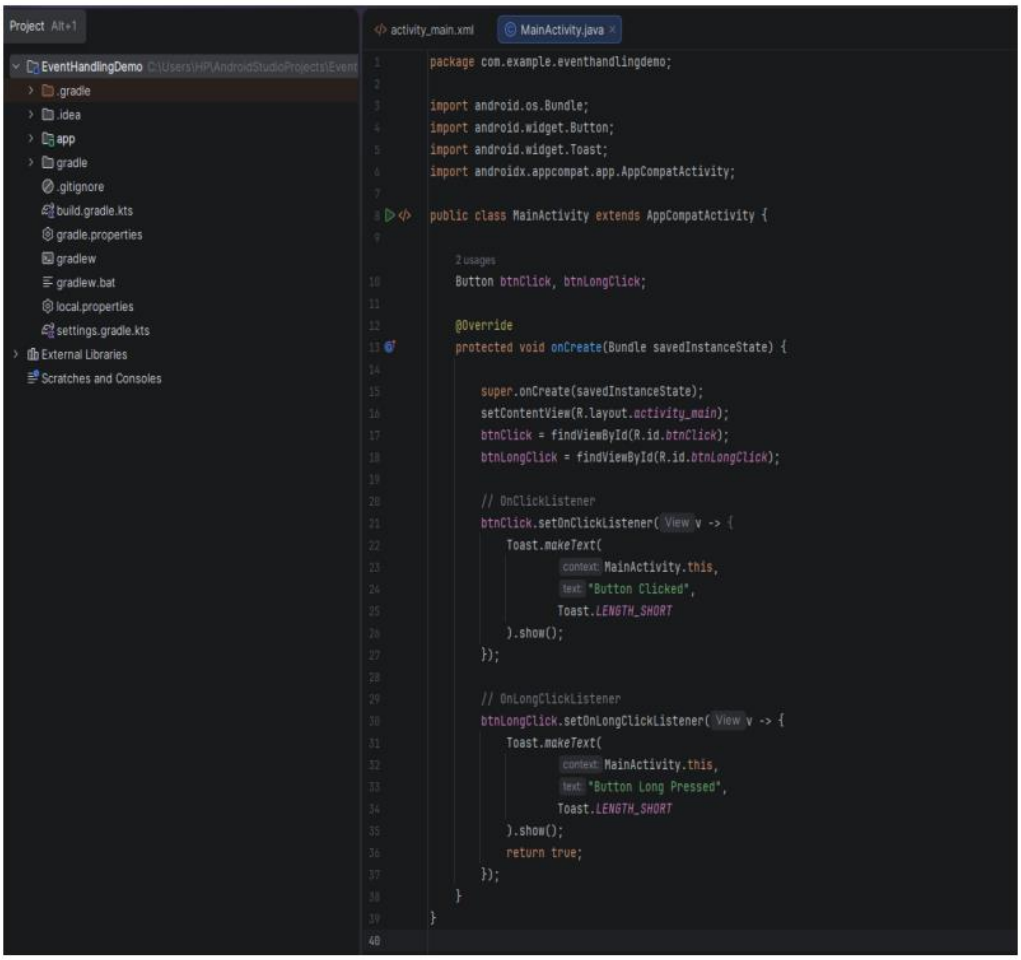
**Activity_main.xml**

Replace code with:

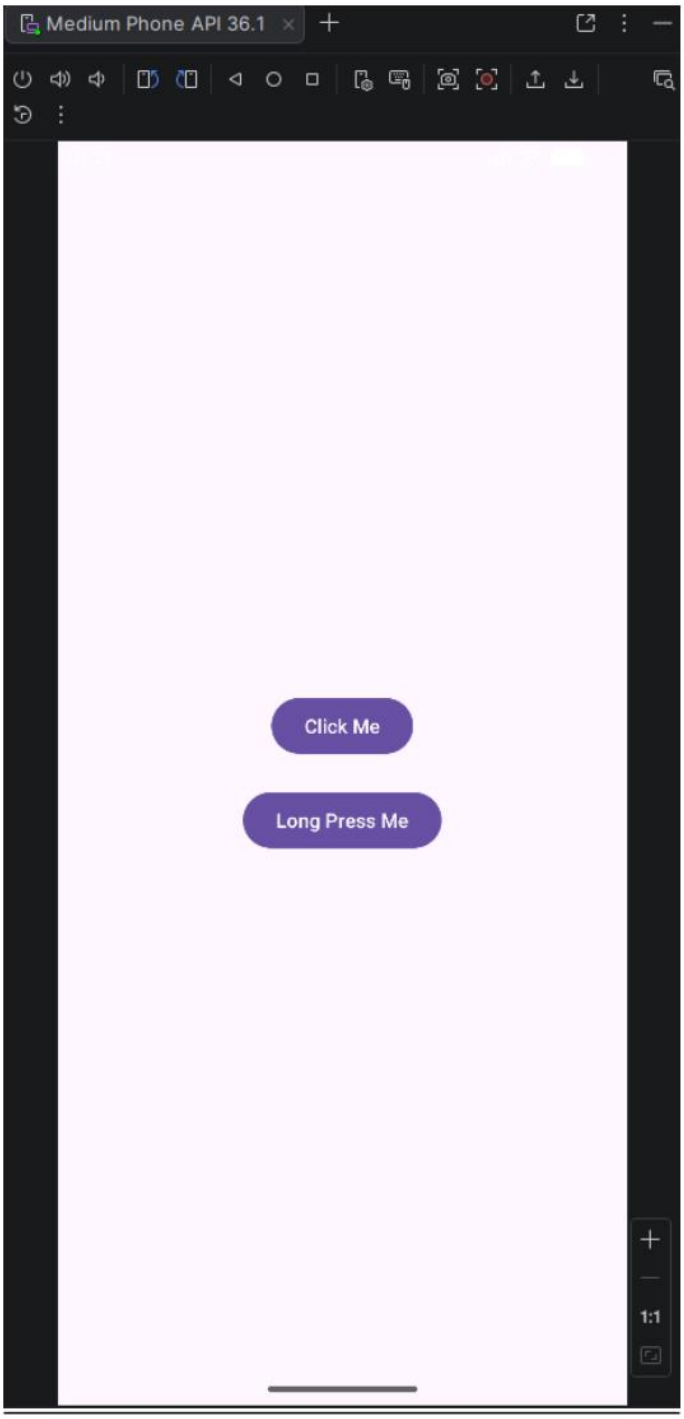



### Step 3: Modify Mainactivity.java.

Replace Code with:




## Step 4: Run The Application.


Medium Phone API 36.1
Click Me
Long Press Me
1:1

# PRACTICAL 6

**DEVELOP AN ANDROID APPLICATION USING ADAPTERS TO POPULATE SPINNER OR LISTVIEW DYNAMICALLY.**

**INTENTS, EVENTS AND LISTENERS**

**Step 1: Create New Android Project.**

Create the Project with **Empty Views Activity** with all details as previously done.

Name the Project: **AdapterDemo**

**Note:**An **Adapter** connects: Data ↔ UI Component. It helps display array data in:

1. Spinner.
2. ListView.
3. GridView.

**Spinner** is a dropdown list used for selecting one option.

**ListView** displays items in a vertical scrollable list.

**Step 2: Design User Interface.**

**Open:activity_main.xml**

Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <Spinner
        android:id="@+id/spinner"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

## Step 3: Modify MainActivity.java.

**Open:**

**java → MainActivity.java**

Replace code with:

```
package com.example.adapterdemo;

import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    Spinner spinner;
    ListView listView;

    String[] cities = {
        "Mumbai",
        "Pune",
        "Delhi",
        "Chennai",
        "Hyderabad"
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        spinner = findViewById(R.id.spinner);
        listView = findViewById(R.id.listView);

        // Spinner Adapter
        ArrayAdapter<String> spinnerAdapter =
            new ArrayAdapter<>(
                this,
                android.R.layout.simple_spinner_dropdown_item,
                cities
            );
        spinner.setAdapter(spinnerAdapter);

        // ListView Adapter
        ArrayAdapter<String> listAdapter =
            new ArrayAdapter<>(
                this,
                android.R.layout.simple_list_item_1,
                cities
            );
        listView.setAdapter(listAdapter);
```

## Step 4: Spinner Selection Handling.

Add the following code after listView.setAdapter(listAdapter); inside onCreate():

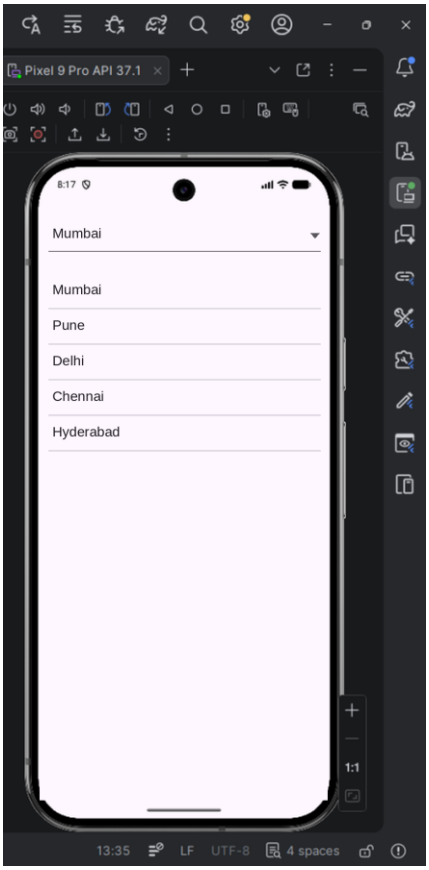
```
        spinner.setOnItemSelectedListener(
            new android.widget.AdapterView.OnItemSelectedListener() {
```

```
            @Override
            public void onItemSelected(
                android.widget.AdapterView<?> parent,
                android.view.View view,
                int position,
                long id) {

                Toast.makeText(
                    MainActivity.this,
                    "Selected : " + cities[position],
                    Toast.LENGTH_SHORT
                ).show();
            }

            @Override
            public void onNothingSelected(
                android.widget.AdapterView<?> parent) {
            }
        });
    }
}
```

**Step 5: Run the Application.**



# Practical 8: Develop an Android Application Implementing Notifications Using Notification Manager

## Step 1: Create New Android Project.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project: ***NotificationDemo***

**Note:**

***A Notification*** *is a message displayed outside the application to inform the user about: New message, Download completion, Updates and Alerts. Notifications appear in Notification Panel.*

***NotificationManager*** *is a system service used to:*

1. *Create notifications.*
2. *Display notifications.*
3. *Manage notifications.*

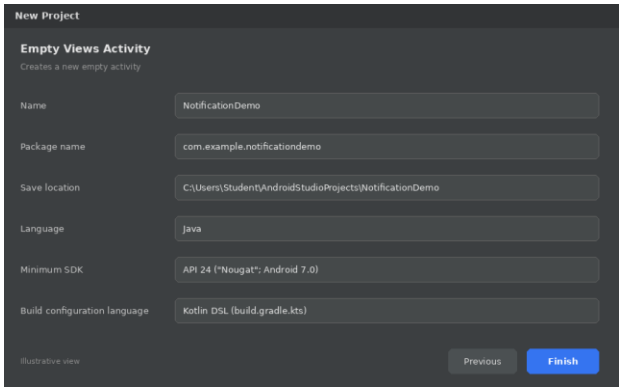



*Figure 1: Creating the NotificationDemo project*

## Step 2: Design User Interface.

**Open: *activity_main.xml***

Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center">

<Button
    android:id="@+id/btnNotify"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
```

```
    android:text="Show Notification"/>

</LinearLayout>
```

## Step 3: Add Permission in Manifest.

**Open: *AndroidManifest.xml***

Add above <application>:

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
```

## Step 4: Modify MainActivity.java.

Replace code with:

```
package com.example.notificationdemo;

import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;

public class MainActivity extends AppCompatActivity {

Button btnNotify;
String CHANNEL_ID = "MyChannel";

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnNotify = findViewById(R.id.btnNotify);

// Button Click
btnNotify.setOnClickListener(v -> {
showNotification();
});
}

private void showNotification() {
NotificationManager manager =
(NotificationManager)
getSystemService(
NOTIFICATION_SERVICE);

// Notification Channel
if (Build.VERSION.SDK_INT
>= Build.VERSION_CODES.O) {
```

```
NotificationChannel channel =
new NotificationChannel(
CHANNEL_ID,
"My Channel",
NotificationManager
.IMPORTANCE_DEFAULT);
manager.createNotificationChannel(channel);
}

// Notification Builder
NotificationCompat.Builder builder =
new NotificationCompat.Builder(
this,
CHANNEL_ID)
.setSmallIcon(
android.R.drawable.ic_dialog_info)
.setContentTitle(
"Notification Example")
.setContentText(
"Hello from Android Notification")
.setAutoCancel(true);

// Show Notification
manager.notify(1, builder.build());
}
}
```

## Step 5: Explanation of Important Components.

| Component | Purpose |
|---|---|
| *NotificationManager* | Manages notifications |
| *NotificationChannel* | Required for Android 8+ |
| *NotificationCompat.Builder* | Builds notification |
| *notify()* | Displays notification |

## Step 6: Run the Application.

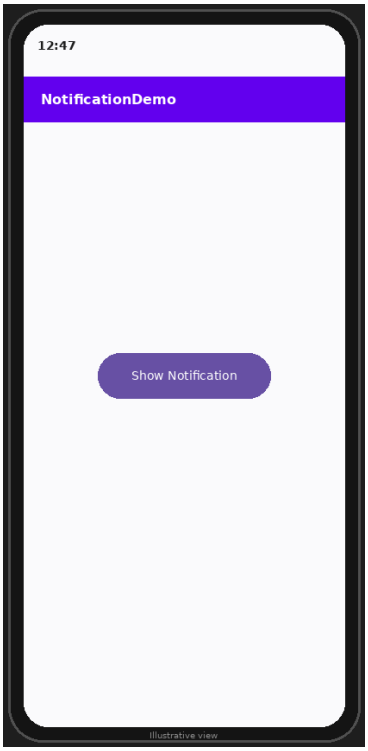



*Figure 5: Running NotificationDemo in the emulator*

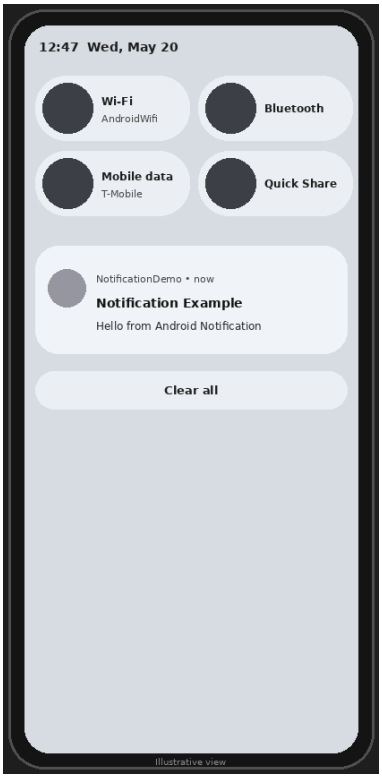



*Figure 6: Output – notification shown in the notification panel*

## Step 7: Grant Notification Permission in Emulator.

i) If app ask for permission for notification, Grant it OR

ii) Select the notification app from the listed apps, from the options menu Grant the Permission for Notification.

# Practical 9: Develop an Android Application Implementing Static and Dynamic Broadcast Receivers

## Step 1: Create New Android Project.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project: ***BroadcastDemo***

**Note:**

***A Broadcast Receiver*** *is an Android component used to receive: System Broadcast Messages.*

**Examples:**

Battery Low, Airplane Mode Changed, SMS Received and Charger Connected

**Types of Broadcast Receivers:**

| Type | Registration Method | Working Condition |
|---|---|---|
| *Static Broadcast Receiver* | Registered in AndroidManifest.xml | Works even app is not running |
| *Dynamic Broadcast Receiver* | Registered using Java code | Works only when app is running |

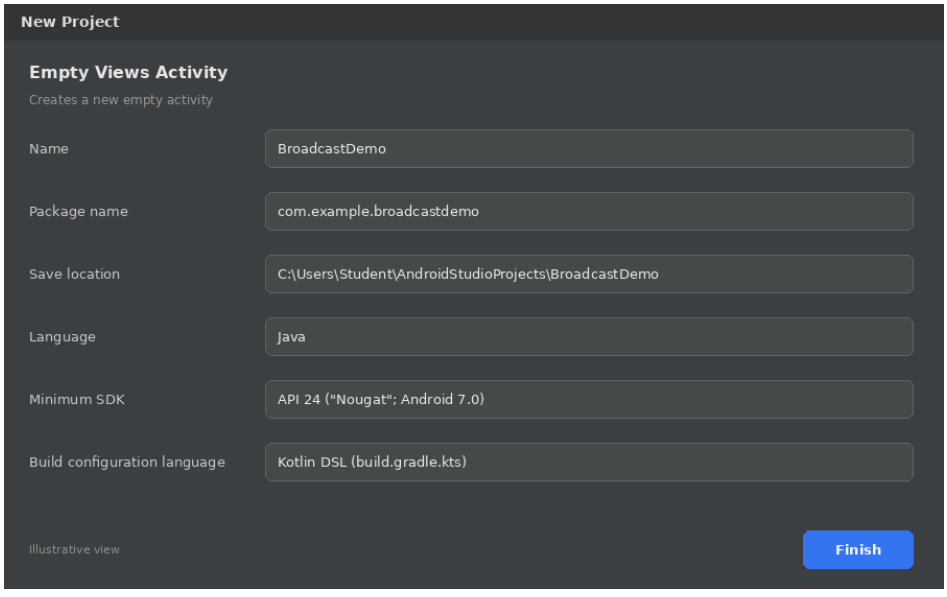



*Figure 1: Creating the BroadcastDemo project*

## Step 2: Design User Interface.

**Open: *activity_main.xml***

Replace code with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center">

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Broadcast Receiver Example"
    android:textSize="22sp"/>

</LinearLayout>
```

## Step 3: Create Broadcast Receiver Class.

**Right Click: *java package → New → Java Class***

**Class Name: *MyReceiver***




*Figure 3: Creating the MyReceiver class*

Replace code with:

```
package com.example.broadcastdemo;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;

public class MyReceiver extends BroadcastReceiver {

@Override
public void onReceive(
Context context,
```

```
Intent intent) {
Toast.makeText(
context,
"Broadcast Received",
Toast.LENGTH_SHORT
).show();
}
}
```

## Step 4: Static Broadcast Receiver.

**Open: *AndroidManifest.xml***

Inside: ***<application>***

add:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.BroadcastDemo">

        <receiver android:name=".MyReceiver" android:exported="true">
            <intent-filter>
                <action
                    android:name=
                    "android.intent.action.AIRPLANE_MODE"/>
            </intent-filter>
        </receiver>

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

*Figure 5: AndroidManifest.xml with the receiver added*

## Step 5: Dynamic Broadcast Receiver.

Modify ***MainActivity.java***

Replace code with:

```
package com.example.broadcastdemo;

import android.content.IntentFilter;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

MyReceiver myReceiver = new MyReceiver();

@Override
protected void onCreate(
Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

// Dynamic Receiver
IntentFilter filter = new IntentFilter();

filter.addAction("android.intent.action.AIRPLANE_MODE");

registerReceiver(myReceiver, filter);
}

@Override
protected void onDestroy() {
super.onDestroy();

unregisterReceiver(myReceiver);
}
}
```

*Figure 6: MainActivity.java*

## Step 5: Explanation of Important Methods.

| Method | Purpose |
|---|---|
| *BroadcastReceiver* | Receives broadcasts |
| *onReceive()* | Executes when broadcast received |
| *registerReceiver()* | Registers dynamic receiver |
| *unregisterReceiver()* | Removes dynamic receiver |

## Step 6: Run the Application.




*Figure 7: Running BroadcastDemo in the emulator*

## Step 7: Testing the Broadcast Receiver.

*Turn:* **Airplane Mode ON/OFF**

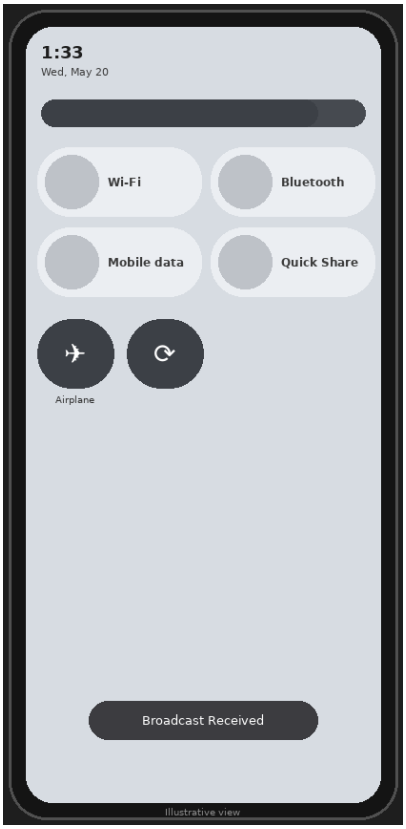



*Figure 8: Output – "Broadcast Received" message after toggling Airplane Mode*

# Practical 10: Develop an Android Application Implementing SQLite Database CRUD Operations

## Step 1: Create New Android Project.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project: ***SQLiteCRUDDemo***

**Note:**

***SQLite*** *is an inbuilt lightweight database in Android used for: Storing application data, Managing records locally and Offline data storage.*

***CRUD Operation Meaning:***

1. ***Create****: Insert new data.*
2. ***Read****: View data.*
3. ***Update****: Modify existing data.*
4. ***Delete****: Remove data.*

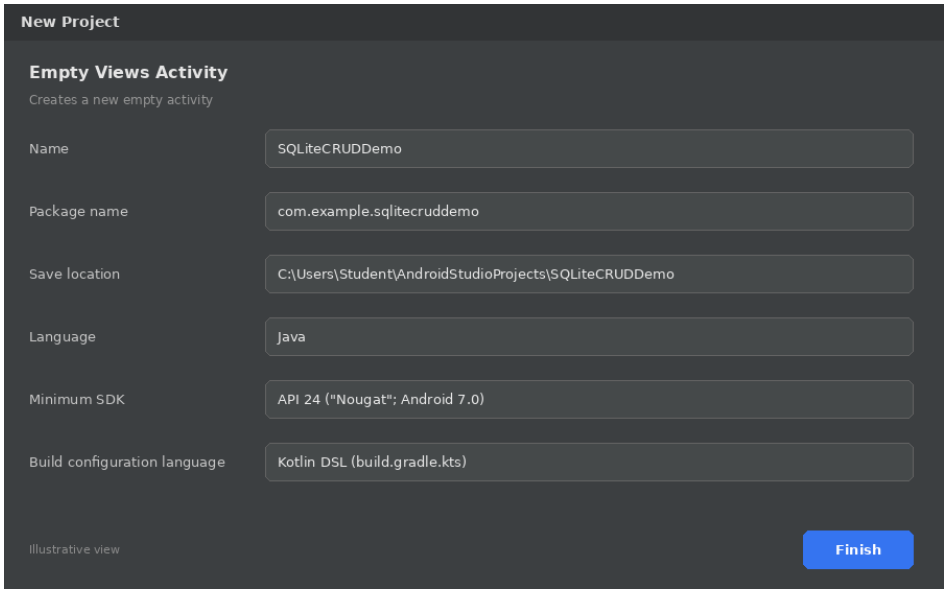



*Figure 1: Creating the SQLiteCRUDDemo project*

## Step 2: Design User Interface.

**Open:** *activity_main.xml*

Replace code with:

```xml
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">

<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name"/>

<EditText
    android:id="@+id/etNewName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter New Name"/>

<Button
    android:id="@+id/btnInsert"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Insert"/>

<Button
    android:id="@+id/btnView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="View Data"/>

<Button
    android:id="@+id/btnUpdate"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Update"/>

<Button
    android:id="@+id/btnDelete"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Delete"/>

<TextView
    android:id="@+id/txtResult"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:paddingTop="20dp"/>
</LinearLayout>
```

## Step 3: Create Database Helper Class.

**Right Click:** ***java package → New → Java Class***

**Class Name: *DBHelper***




*Figure 3: Creating the DBHelper class*

Replace code with:

```
package com.example.sqlitecruddemo;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class DBHelper extends SQLiteOpenHelper {

public DBHelper(Context context) {
super(context,
"studentDB",
null,
1);
}

@Override
public void onCreate(SQLiteDatabase db) {
db.execSQL("CREATE TABLE student(" + "name TEXT)");
}

@Override
public void onUpgrade(SQLiteDatabase db,
int oldVersion,
int newVersion) {
}

// Insert
public void insertData(String name) {
SQLiteDatabase db = this.getWritableDatabase();
ContentValues values = new ContentValues();
values.put("name", name);
db.insert(
"student",
null,
values);
```

```
}

// View
public String viewData() {
SQLiteDatabase db = this.getReadableDatabase();
Cursor cursor = db.rawQuery( "SELECT * FROM student", null);
String data = "";
while(cursor.moveToNext()) {
data += cursor.getString(0)
+ "\n";
}
return data;
}

// Update
public void updateData(String oldName,
String newName) {
SQLiteDatabase db = this.getWritableDatabase();
ContentValues values = new ContentValues();
values.put("name", newName);
db.update(
"student",
values,
"name=?",
new String[]{oldName});
}

// Delete
public void deleteData(String name) {
SQLiteDatabase db = this.getWritableDatabase();
db.delete(
"student",
"name=?",
new String[]{name});
}
}
```

## Step 4: Modify MainActivity.java.

Replace code with:

```
package com.example.sqlitecruddemo;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

EditText etName, etNewName;
TextView txtResult;

Button btnInsert, btnView, btnUpdate, btnDelete;
```

```
DBHelper dbHelper;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);

etName = findViewById(R.id.etName);
etNewName = findViewById(R.id.etNewName);
txtResult = findViewById(R.id.txtResult);
btnInsert = findViewById(R.id.btnInsert);
btnView = findViewById(R.id.btnView);
btnUpdate = findViewById(R.id.btnUpdate);
btnDelete = findViewById(R.id.btnDelete);

dbHelper = new DBHelper(this);

// Insert
btnInsert.setOnClickListener(v -> {
String name = etName.getText().toString();

dbHelper.insertData(name);
Toast.makeText(
this,
"Data Inserted",
Toast.LENGTH_SHORT
).show();
});

// View
btnView.setOnClickListener(v -> {
String data = dbHelper.viewData();
txtResult.setText(data);
});

// Update
btnUpdate.setOnClickListener(v -> {
String oldName = etName.getText().toString();

String newName = etNewName.getText().toString();

dbHelper.updateData( oldName, newName);

Toast.makeText(
this,
"Data Updated",
Toast.LENGTH_SHORT
).show();
});

// Delete
btnDelete.setOnClickListener(v -> {
dbHelper.deleteData(
etName.getText().toString());

Toast.makeText(
```

```
this,
"Data Deleted",
Toast.LENGTH_SHORT
).show();
});
}
}
```

## Step 5: Explanation of Important Methods.

| Method | Purpose |
|---|---|
| *insert()* | Inserts data |
| *rawQuery()* | Reads data |
| *update()* | Updates records |
| *delete()* | Deletes records |
| *SQLiteOpenHelper* | Database management |

## Step 6: Run the Application.

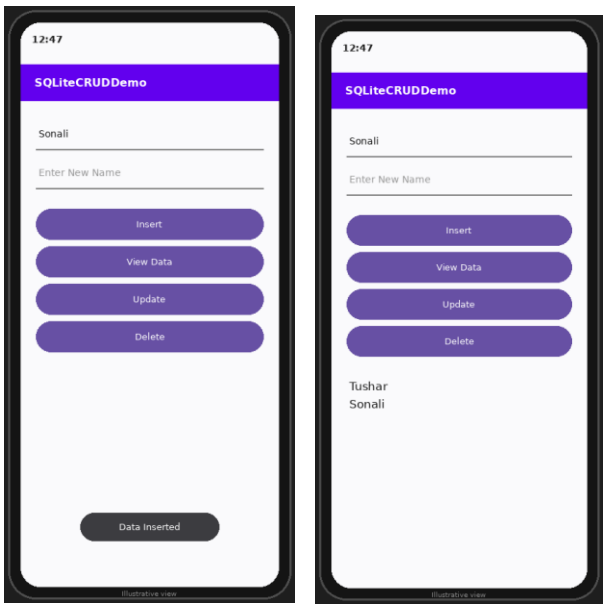



*Figure 6: Insert ("Data Inserted") and View Data*

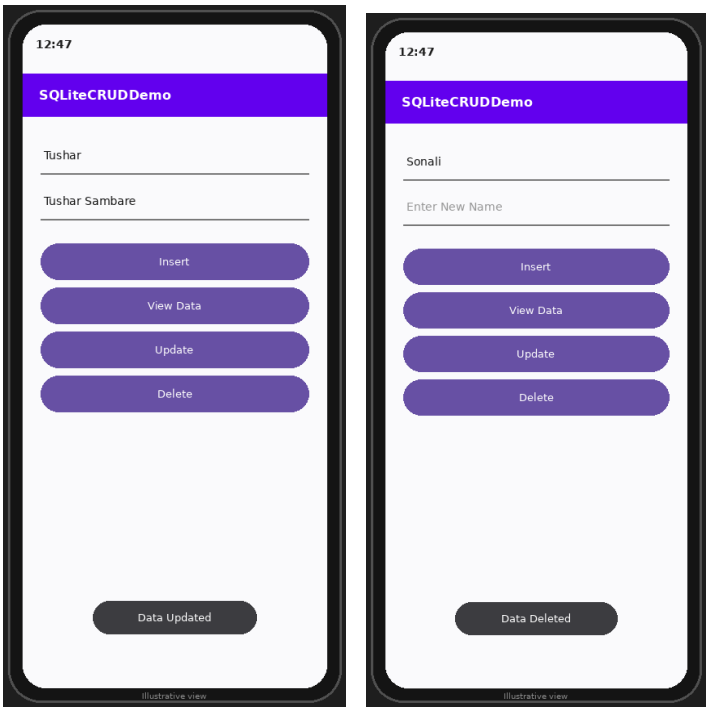



*Figure 7: Update ("Data Updated") and Delete ("Data Deleted")*

# PRACTICAL 11

## Develop An Android Application Integration SQLITE With ListView Or RecyclerView.

### STEP1 : : CREATE NEW ANDROID PROJECT.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project : SQLiteCRUDDemo

### **STEP2 : Design User Interface.**

Open : activity_main.xml

DBhelper.java | activity_main.xml | MainActivity.java

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".MainActivity">
    <EditText
        android:id="@+id/editTextName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Name"
        android:inputType="textPersonName" />
    <EditText
        android:id="@+id/editTextRoll"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="Enter Roll Number"
        android:inputType="number" />
    <Button
        android:id="@+id/buttonInsert"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="Insert Data" />
    <Button
        android:id="@+id/buttonShow"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Show Inserted Data" />
    <ListView
        android:id="@+id/listView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
```

## STEP 3: Create Database Helper Class.

Create New Java Class in Program Folder , Name as : **DBHelper**

DBhelper.java | MainActivity.java

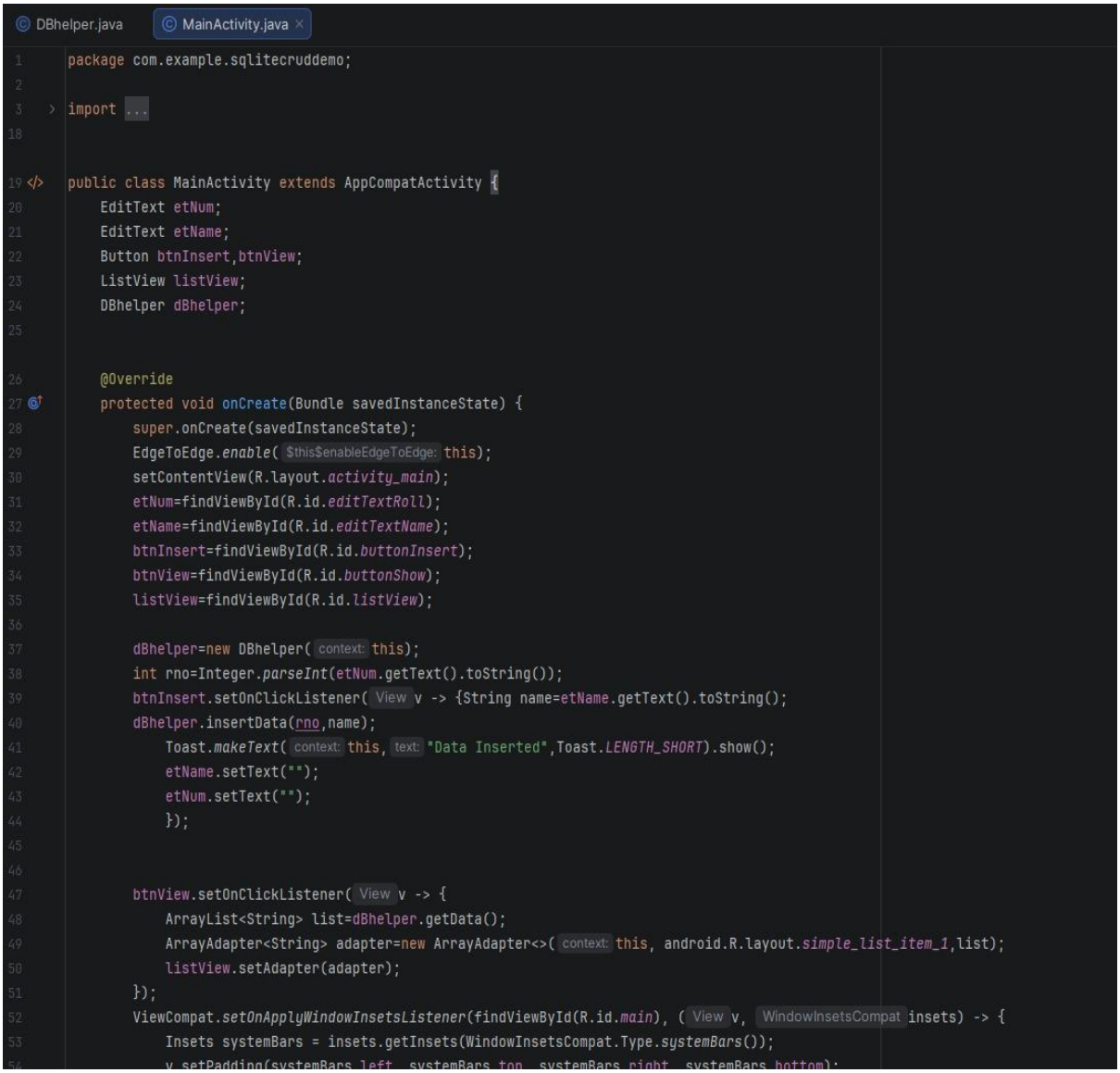
```java
package com.example.sqlitecruddemo;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

import java.util.ArrayList;

2 usages
public class DBhelper extends SQLiteOpenHelper {
    1 usage
    public DBhelper(Context context) { super(context, name: "StudentDB", factory: null, version: 1); }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE student(rno INTEGER,name TEXT)");
    }
    10 usages
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    }
    1 usage
    public void insertData(int rno ,String name){
        SQLiteDatabase db=this.getWritableDatabase();
        ContentValues values=new ContentValues();
        values.put("name",name);
        values.put("Roll Number",rno);
        db.insert( table: "Student", nullColumnHack: null,values);
    }
    1 usage
    public ArrayList<String> getData(){
        ArrayList<String>list=new ArrayList<>();
        SQLiteDatabase db=this.getReadableDatabase();
        Cursor cursor=db.rawQuery( sql: "SELECT*FROM student", selectionArgs: null);
        while (cursor.moveToNext()){
            list.add(cursor.getString( columnIndex: 0)+"-"+ cursor.getInt( columnIndex: 1));
        }
        return  list;
    }
}
```

# STEP 4 : Modify MainActivity.java.



DBhelper.java | MainActivity.java

```
package com.example.sqlitecruddemo;

import ...


public class MainActivity extends AppCompatActivity {
    EditText etNum;
    EditText etName;
    Button btnInsert,btnView;
    ListView listView;
    DBhelper dBhelper;

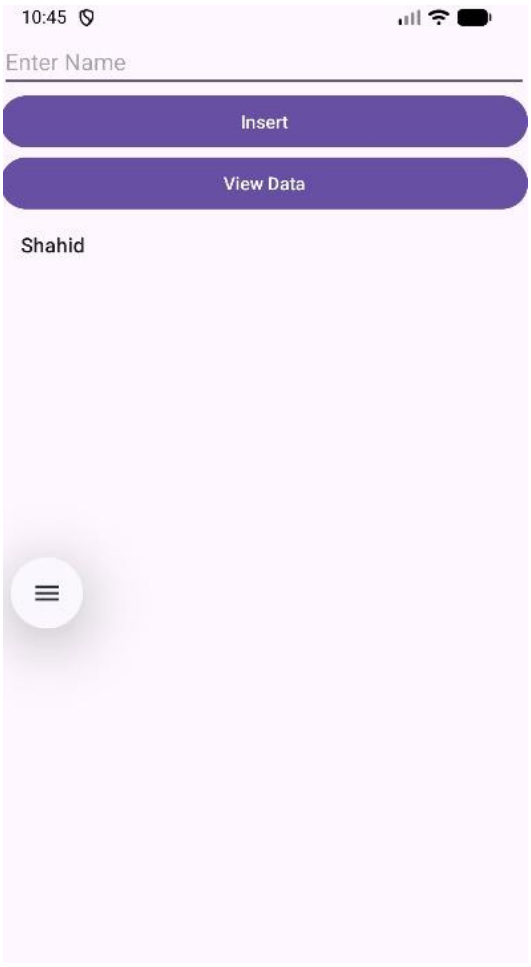

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        etNum=findViewById(R.id.editTextRoll);
        etName=findViewById(R.id.editTextName);
        btnInsert=findViewById(R.id.buttonInsert);
        btnView=findViewById(R.id.buttonShow);
        listView=findViewById(R.id.listView);

        dBhelper=new DBhelper( context: this);
        int rno=Integer.parseInt(etNum.getText().toString());
        btnInsert.setOnClickListener( View v -> {String name=etName.getText().toString();
        dBhelper.insertData(rno,name);
            Toast.makeText( context: this, text: "Data Inserted",Toast.LENGTH_SHORT).show();
            etName.setText("");
            etNum.setText("");
            });


        btnView.setOnClickListener( View v -> {
            ArrayList<String> list=dBhelper.getData();
            ArrayAdapter<String> adapter=new ArrayAdapter<>( context: this, android.R.layout.simple_list_item_1,list);
            listView.setAdapter(adapter);
        });
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
```

```
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });
}
}
```

STEP 5 : Run The Application



# PRACTICAL 13

Develop An Android Application Demonstration Threads And For Background Processing.

## STEP1 : : CREATE NEW ANDROID PROJECT.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project : ThreadHandlerDemo

## STEP2 : Design User Interface.

Open : activity_main.xml

activity_main.xml | MainActivity.java

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Background Task Complete"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="@id/button" />

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Background Task"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

## STEP 3: Modify MainActivity.java.

activity_main.xml | MainActivity.java

```
package com.example.mythread;

import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.os.Message;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);

        Button btn= findViewById(R.id.button);
        TextView txt= findViewById(R.id.textView);

        final Handler uiHandler = new Handler(Looper.getMainLooper()){
            @Override
            public void handleMessage(@NonNull Message msg) {
                super.handleMessage(msg);
                txt.setText("Text Finished Successfully!");
            }
        };

btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        txt.setText("Working...");

```

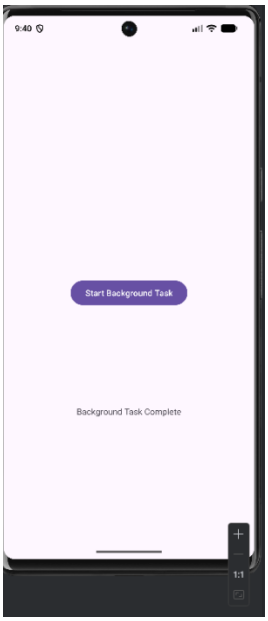
```
        Thread workerThread = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    Thread.sleep( millis: 5000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }

                uiHandler.sendEmptyMessage( what: 0);
            }
        });

        workerThread.start();
    }
});

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v,  WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}
```

STEP 4 : Run The Application



# PRACTICAL 14

Develop An Android Application Implementing Media APIs And Telephone APIs.

## STEP1 : : CREATE NEW ANDROID PROJECT.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project : MediaTelephoneDemo

## STEP2 : Design User Interface.

Open : : activity_main.xml

activity_main.xml | MainActivity.java

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Media Song"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.322" />
    <Button
        android:id="@+id/button2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Button Phone"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

## STEP3 : Add Audio File.

Create New folder in Android Project Panel : res ->raw

Download or use pre-existing small size MP3 file inside raw folder : music.mp3

## STEP4 :Modify MainActivity.java.

activity_main.xml | MainActivity.java

```java
package com.example.mediaapi;
import android.content.Intent;
import android.media.MediaPlayer;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

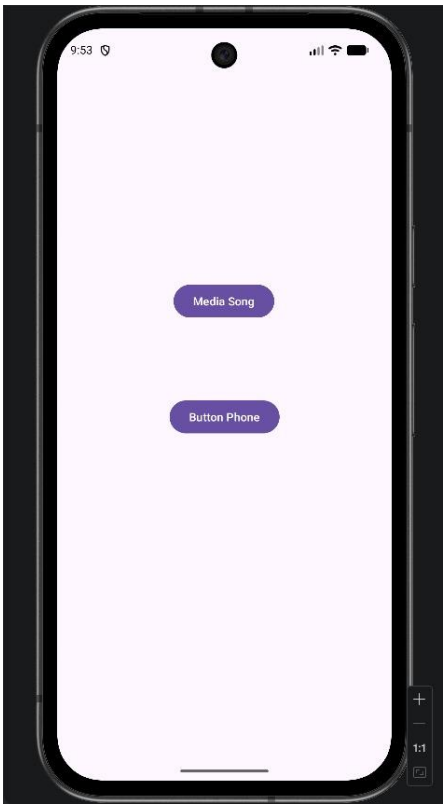
3 usages
public class MainActivity extends AppCompatActivity {
    8 usages
    MediaPlayer mp;

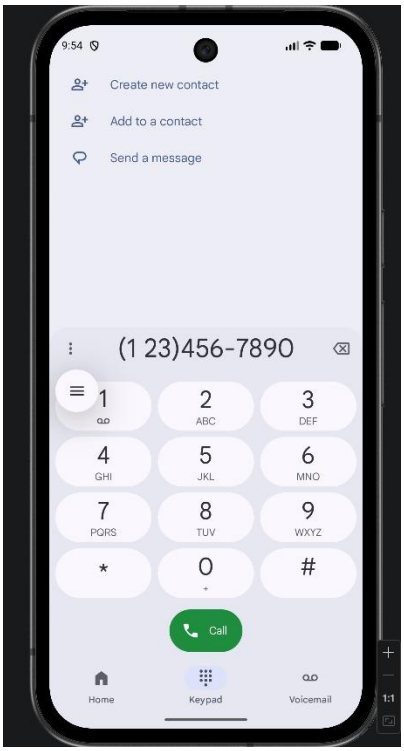
    no usages
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        Button btn=findViewById(R.id.button);
        btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if ((mp!=null)&& mp.isPlaying()){
                    mp.pause();
                } else {
                    if (mp==null)
                    {
                        mp=MediaPlayer.create( context: MainActivity.this,R.raw.secondm);
                    }
                }
                mp.start();
            }
        });
```

```
        Button btnphone=findViewById(R.id.button2);
        btnphone.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent=new Intent(Intent.ACTION_DIAL);
                intent.setData(Uri.parse( uriString: "tel:" + "8591851103"));
                startActivity(intent);
            }
        });
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }

    no usages
    @Override
    protected void onStop() {
        if (mp!=null) mp.release();
        super.onStop();
    }
}
```

## STEP 5 : Run The Application



PRACTICAL 15

# Develop An Android Application Implementing Runtime Permission And Security Handling In AndroidManifest.Xml

## STEP1 : : CREATE NEW ANDROID PROJECT.

Create the Project with Empty Views Activity with all details as previously done.

Name the Project : **PermissionDemo**

## STEP2 :Add Permission in ANdroidManifest.xml .

Open : : AndroidManifest.xml

activity_main.xml | MainActivity.java | AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">



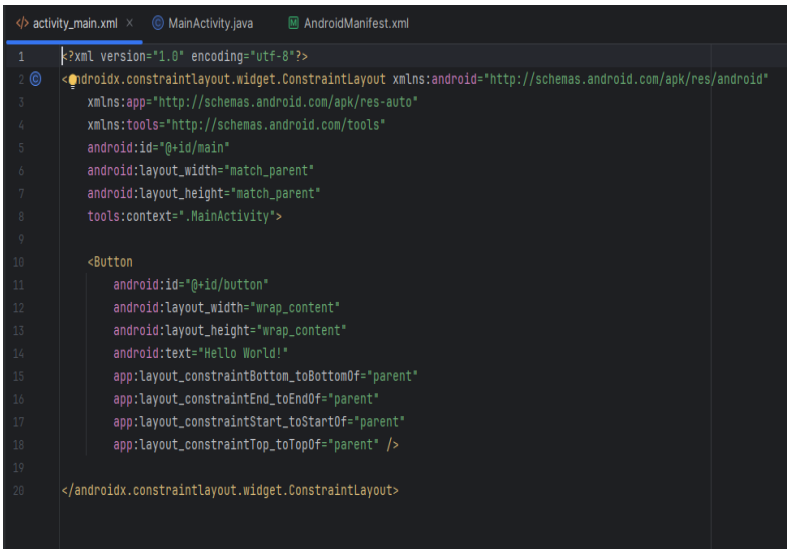
    <uses-feature android:name="android.hardware.camera"
        android:required="false"/>
    <uses-permission android:name="android.permission.CAMERA"/>
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="MyAppPermission"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyAppPermission">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

## STEP3 : Design User Interface.

## Open : : activity_main.xml

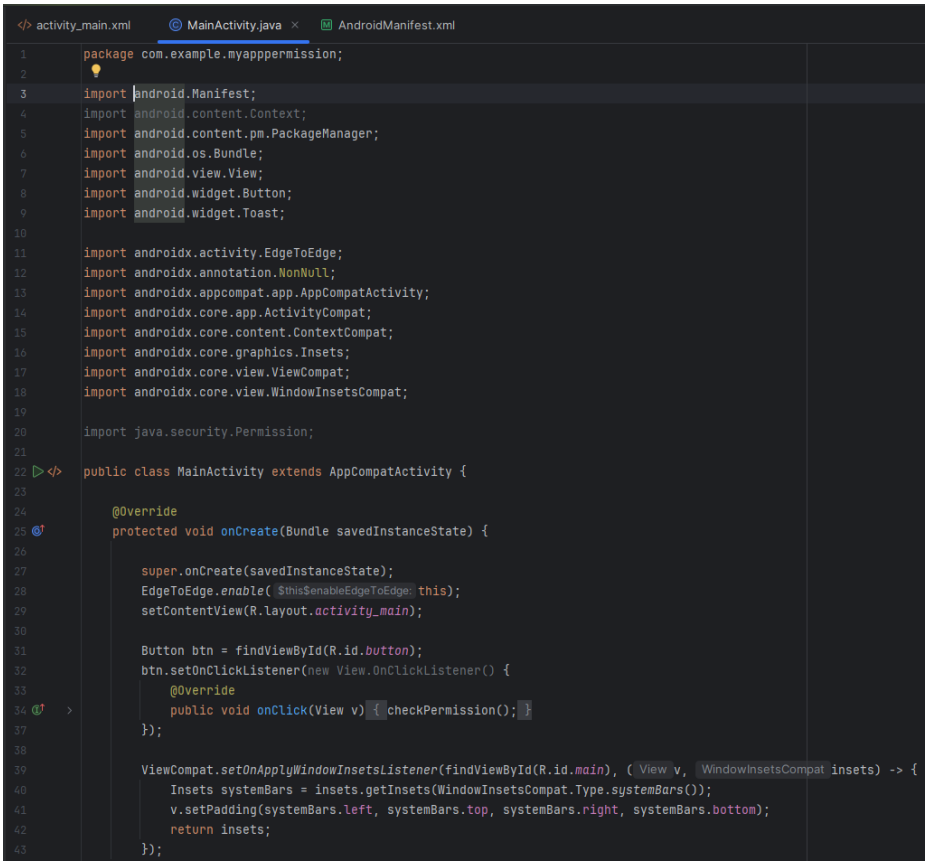


```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

## STEP4 :Modify MainActivity.java.



```
package com.example.myapppermission;

import android.Manifest;
import android.content.Context;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import java.security.Permission;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);

        Button btn = findViewById(R.id.button);
        btn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) { checkPermission(); }
        });

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
```

```
    }

    1 usage
    public void checkPermission() {
        if (ContextCompat.checkSelfPermission( context: this, Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED) {
            Toast.makeText( context: this,  text: "Permission Already granted", Toast.LENGTH_LONG).show();
        } else {
            ActivityCompat.requestPermissions( activity: this, new String[]{Manifest.permission.CAMERA},  requestCode: 1);
        }
    }

    @Override
    public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == 1) {
            if(grantResults[0]==PackageManager.PERMISSION_GRANTED) {
                Toast.makeText( context: this,  text: "Permissin Granted", Toast.LENGTH_SHORT).show();
                ActivityCompat.requestPermissions( activity: this, new String[]{Manifest.permission.CAMERA},  requestCode: 1);
            }
            else {
                Toast.makeText( context: this,  text: "Permissin Denied", Toast.LENGTH_SHORT).show();
                ActivityCompat.requestPermissions( activity: this, new String[]{Manifest.permission.CAMERA},  requestCode: 1);
            }
        }

    }
}
```

## STEP 5 : Run The Application

